

WSL + Python 環境構築マニュアル

Windows で始める Linux 入門と
Python 仮想環境の基礎

対象：Windows ユーザー向け Python 入門者

レベル：コマンドライン未経験者から

バージョン：2026 年度版

目次

はじめに	7
第 1 章 なぜ WSL を使うのか	9
1.1 プログラミングの世界と Linux	9
1.2 Windows だと何が不便なのか	9
1.3 WSL とは何か	10
1.4 WSL・仮想マシン・ネイティブ Linux の比較	10
1.5 このマニュアルで使う Linux ディストリビューション	11
第 2 章 環境構築の全体像	12
2.1 これから何をするか	12
2.2 所要時間の目安	13
2.3 つまづきやすいポイント	13
2.4 Windows と Linux の「並走」イメージ	13
第 3 章 WSL の導入	15
3.1 インストール前の確認	15
3.1.1 Windows のバージョン確認	15
3.1.2 仮想化機能の確認	15
3.2 WSL のインストール	15
3.2.1 PowerShell を管理者権限で起動する	15
3.2.2 WSL インストールコマンドの実行	16
3.2.3 PC を再起動する	16
3.3 Ubuntu の初期設定	17
3.3.1 Ubuntu の初回起動	17
3.3.2 ユーザー名の設定	17
3.3.3 パスワードの設定	17
3.3.4 パッケージの更新	18

3.4	Windows Terminal の設定 (推奨)	18
3.4.1	Windows Terminal のインストール	19
3.4.2	Ubuntu タブの開き方	19
3.5	インストールの確認	19
第 4 章	大学ネットワークのプロキシ設定	20
4.1	プロキシとは・なぜ必要か	20
4.2	設定が必要な箇所の概要	20
4.3	~/.bashrc : ターミナル全般のプロキシ設定	21
4.4	/etc/environment : システム全体のプロキシ設定	22
4.5	/etc/apt/apt.conf : apt 専用のプロキシ設定	22
4.6	git のプロキシ設定	23
第 5 章	ターミナルの基本操作	25
5.1	ターミナルとは何か	25
5.2	プロンプトの読み方	25
5.3	よく使うショートカットキー	26
5.3.1	コマンド実行・キャンセル	26
5.3.2	カーソル移動	27
5.3.3	履歴・補完	27
5.3.4	画面操作	27
5.4	テキストのコピーとペースト	28
5.5	コマンドの基本構造	28
第 6 章	Unix/Linux の考え方に慣れる	29
6.1	ファイルシステムとは	29
6.2	パスとは何か	29
6.3	重要なパス記号	29
6.3.1	/ (ルートディレクトリ)	30
6.3.2	~ (チルダ: ホームディレクトリ)	30
6.3.3	. (ドット: カレントディレクトリ)	30
6.3.4	.. (ドット 2 つ: 親ディレクトリ)	31
6.4	絶対パスと相対パス	31
6.4.1	絶対パス (absolute path)	31
6.4.2	相対パス (relative path)	31
6.5	大文字・小文字の区別	32
6.6	Windows のファイルへ WSL からアクセスする	32
6.7	ファイルとディレクトリ	33

6.8	パーミッション（権限）の概念	33
第 7 章	基本コマンドを使ってみよう	34
7.1	pwd — 今いる場所を確認する	34
7.2	ls — ディレクトリの内容を表示する	34
7.2.1	よく使うオプション	35
7.3	cd — ディレクトリを移動する	36
7.4	mkdir — ディレクトリを作成する	36
7.5	touch — 空のファイルを作成する	37
7.6	cp — ファイルやディレクトリをコピーする	37
7.7	mv — ファイルを移動・名前変更する	38
7.8	rm — ファイルやディレクトリを削除する	38
7.9	cat — ファイルの内容を表示する	39
7.10	まとめ：基本コマンド早見表	40
第 8 章	コマンドの見方と覚え方	41
8.1	コマンドのヘルプを読む	41
8.1.1	--help：簡易ヘルプを表示する	41
8.1.2	man：詳細マニュアルを表示する	41
8.2	補足コマンド集	42
8.2.1	whoami：ログインしているユーザー名を表示する	42
8.2.2	clear：画面をクリアする	42
8.2.3	history：コマンド履歴を表示する	43
8.2.4	which：コマンドの実体がどこにあるか確認する	43
8.2.5	exit：ターミナルを終了する	44
8.3	発展コマンド	44
8.3.1	less：ファイルをページ送りしながら見る	44
8.3.2	head / tail：ファイルの先頭・末尾を表示する	44
8.3.3	wc：行数・単語数・文字数を数える	44
8.4	コマンドの構造を理解する	45
8.5	コマンドを覚えるコツ	45
第 9 章	練習用ディレクトリを作って操作してみよう	46
9.1	練習の前に：現在位置の確認	46
9.2	練習 1：ディレクトリ構造を作る	46
9.3	練習 2：ファイルを作成・確認する	47
9.4	練習 3：ファイルのコピーと移動	48
9.5	練習 4：ディレクトリをまるごとコピー	48

9.6	練習 5：不要ファイルの削除	49
9.7	まとめ演習	49
第 10 章	Python を使う準備	51
10.1	Ubuntu と Python	51
10.2	Python のインストール確認	51
10.3	Python のインストール	52
10.4	pip とは何か	52
10.5	Python の動作確認	53
10.6	Python スクリプトファイルの実行	54
第 11 章	仮想環境とは何か	55
11.1	なぜ仮想環境が必要か	55
11.1.1	依存関係の衝突の例	55
11.1.2	グローバル環境汚染	55
11.1.3	環境の再現が難しくなる	56
11.2	仮想環境の仕組み	56
11.3	仮想環境を使うメリット	56
11.4	このマニュアルでの方針	57
第 12 章	venv を使った標準的な Python 環境構築	58
12.1	venv の基本的な流れ	58
12.2	ステップ 1：プロジェクトディレクトリの作成	58
12.3	ステップ 2：仮想環境の作成	58
12.4	ステップ 3：仮想環境の有効化	59
12.5	ステップ 4：パッケージのインストール	60
12.6	requirements.txt による環境の共有・再現	60
12.7	ステップ 5：仮想環境の無効化	61
12.8	仮想環境の削除	61
12.9	トラブル：仮想環境を有効化したのに (.venv) が出ない	62
12.10	作業フローのまとめ	62
第 13 章	オプション：uv を使った快適な環境構築	63
13.1	uv とは何か	63
13.2	uv のインストール	64
13.3	uv で仮想環境を作る	64
13.4	uv でパッケージを追加する	65
13.5	uv の依存管理ファイル	65

13.6	uv を使う場面と venv を使う場面	66
第 14 章	推奨される作業スタイル	67
14.1	ディレクトリ構成の推奨	67
14.2	毎回の作業フロー	68
14.3	Windows 側ファイルとの使い分け	68
14.4	.gitignore で仮想環境を除外する（発展）	68
14.5	ターミナルを複数使う	69
第 15 章	VS Code と連携する	70
15.1	VS Code とは	70
15.2	VS Code のインストール（Windows 側）	70
15.3	WSL 拡張機能のインストール	70
15.4	WSL から VS Code を開く	71
15.5	Python 拡張機能のインストール	71
15.6	仮想環境のインタープリタを選択する	71
15.7	基本的な VS Code の使い方	72
15.8	Python ファイルの実行（VS Code から）	72
第 16 章	よくある質問とトラブルシューティング	73
16.1	インストール・起動時のトラブル	73
16.1.1	wsl -install でエラーが出る	73
16.1.2	Ubuntu が起動しない・「参照しているコンポーネントが見つからない」	73
16.1.3	「0x80370102」エラー	74
16.2	ターミナル操作のトラブル	74
16.2.1	パスワードを入力しても何も表示されない	74
16.2.2	コマンドが終わらない・止まったように見える	74
16.2.3	Tab 補完が効かない	74
16.3	コマンドのトラブル	75
16.3.1	「command not found」と出る	75
16.3.2	「Permission denied」と出る	75
16.3.3	「No such file or directory」と出る	75
16.4	Python・pip のトラブル	75
16.4.1	「pip: command not found」または「pip3: command not found」	75
16.4.2	pip install でエラーが出る（externally-managed-environment）	76
16.4.3	インストールしたパッケージが見つからない（ImportError）	76
16.4.4	仮想環境の有効化コマンドを入力したが（.venv）が表示されない	76

16.5	その他のトラブル	77
16.5.1	rm で削除したファイルを元に戻したい	77
16.5.2	Windows 側のファイルに WSL からアクセスできない	77
16.5.3	sudo apt update でエラーが出る	77
16.5.4	文字化けする・日本語が表示されない	78
第 17 章	まとめ	79
17.1	このマニュアルで学んだこと	79
17.2	次のステップ	79
17.3	困ったときの参考先	80
17.4	最後に	80
付録 A	よく使うコマンド一覧	81
A.1	ファイル・ディレクトリ操作	81
A.2	ファイル閲覧	82
A.3	システム・情報確認	82
A.4	パッケージ管理 (apt)	83
A.5	Python・仮想環境	83
付録 B	よく使うオプション一覧	85
B.1	ls のオプション	85
B.2	cp のオプション	86
B.3	rm のオプション	86
B.4	mkdir のオプション	86
B.5	head / tail のオプション	87
B.6	wc のオプション	87
付録 C	用語集	88
付録 D	参考資料	90
D.1	公式ドキュメント	90
D.2	Linux コマンドの参考サイト	90
D.3	学習リソース	90
付録 E	演習課題	92
E.1	基礎演習：コマンドライン操作	92
E.2	中級演習：Python 仮想環境	93
E.3	発展演習	94

はじめに

このマニュアルの目的

このマニュアルは、Windows を使っている大学生が「コマンドライン操作」と「Python 仮想環境」を理解して使えるようになることを目的としています。

単なる手順書ではなく、

- なぜその操作が必要なのか
- 何が起きているのか
- どこが間違いやすいか

を丁寧に説明するよう心がけています。コマンドを「呪文のように覚える」のではなく、意味を理解しながら使えるようになることを目指してください。

このマニュアルで学ぶこと

1. **WSL の導入**：Windows 上に Linux 環境 (Ubuntu) を構築する
2. **ターミナルの基本操作**：画面の見方、入力方法、ショートカットを理解する
3. **Unix/Linux の考え方**：ファイルシステム、パスの仕組みを理解する
4. **基本コマンドを使う**：ファイル操作など日常的なコマンドを習得する
5. **Python 仮想環境を構築する**：venv を使って授業ごとに独立した環境を作る
6. **(発展) uv を使う**：より快適な環境管理ツールを試してみる

対象読者

- Windows を日常的に使っているが、Linux はほぼ初めて
- コマンドライン (黒い画面) 操作に慣れていない
- Python を大学の授業や演習で使い始めようとしている
- 「WSL」「ターミナル」「仮想環境」という言葉は聞いたことはある

マニュアルの読み進め方

- 初めの方：第 1 章から順に読み進めてください。
- WSL はインストール済みの方：第 4 章（ターミナルの基本操作）から始めてください。
- コマンドは知っているが Python 環境が不安な方：第 10 章（仮想環境とは何か）から始めてください。
- リファレンスとして使いたい方：付録 A（コマンド一覧）をご活用ください。

マニュアル内の表記について

このマニュアルでは、以下の表記を使います。

表記	意味
\$ コマンド	Ubuntu/WSL ターミナルで入力するコマンド
> コマンド	PowerShell (Windows) で入力するコマンド
イタリック部分	実際の値に置き換える箇所
<code>Ctrl</code> + <code>C</code>	キーボードのキー操作（同時押し）

メモ

コマンド例の先頭にある \$ や > は「プロンプト」と呼ばれる記号で、**実際には入力しません**。\$ 以降の部分だけを入力してください。

注意

このマニュアルでは、コマンドを **Ubuntu 側 (WSL) で実行するか、Windows 側 (PowerShell) で実行するか**を明確に区別しています。混同すると動作しないことがあります。必ず確認してから実行してください。

必要な環境

- Windows 10 バージョン 2004（ビルド 19041）以降、または Windows 11
- インターネット接続
- 管理者権限（インストール時のみ必要）
- ディスク空き容量：5 GB 以上（推奨）

第1章 なぜ WSL を使うのか

1.1 プログラミングの世界と Linux

大学の授業や研究、そして将来の仕事でプログラミングをする機会が増えると、「Linux」や「Unix」という言葉を目にするようになります。

Linux はサーバーや研究用 PC、スーパーコンピュータ、クラウド環境など、**開発・研究の現場で圧倒的に使われているオペレーティングシステム (OS)** です。例えば、次のような場面で Linux が使われています。

- 大学や研究機関の計算サーバー
- Web サービスのサーバー (Google、Amazon など)
- Docker などのコンテナ技術
- 機械学習・深層学習の実験環境 (GPU クラスタなど)
- Raspberry Pi などの組み込みシステム

Python の多くのライブラリやツールは、まず Linux 向けに開発されます。そのため、「Linux のコマンドラインに慣れておく」ことは、理工系の大学生にとって**非常に重要なスキル**です。

1.2 Windows だと何が不便なのか

Windows は日常の利用にとっても便利ですが、プログラミング環境としてはいくつかの不便さがあります。

- パスの区切り文字が違う：Windows は C:\Users\、Linux は /home/ を使う
- コマンドが違う：Linux の ls、grep、chmod 等のコマンドが使えない
- ライブラリのインストールが複雑：一部の Python パッケージは Windows では設定が難しい
- シェルスクリプトが動かない：.sh などのスクリプトが標準では動かない

ポイント

「コマンドライン」や「シェル」とは、マウスではなくキーボードで文字を打ってコンピュータを操作する仕組みのことです。「黒い画面」と呼ばれることもあります。

1.3 WSL とは何か

WSL (Windows Subsystem for Linux) は、Windows の中で Linux を動かす仕組みです。2016 年にマイクロソフトが公式に提供を始め、現在は「WSL2」というより高性能な版が標準となっています。

WSL を使うと、Windows を使いながら、ターミナルで Linux のコマンドを実行したり、Linux 用のソフトウェアをインストールしたりできます。

WSL でできること	説明
Linux コマンドの実行	ls, grep, chmod など Linux 標準コマンドが使える
Ubuntu ソフトのインストール	apt コマンドでパッケージを簡単に追加できる
Python 環境の構築	Linux ネイティブな Python 環境が作れる
シェルスクリプトの実行	.sh ファイルをそのまま実行できる
VS Code との連携	Windows の VS Code から WSL の環境を操作できる

1.4 WSL・仮想マシン・ネイティブ Linux の比較

Linux を使う方法はいくつかあります。それぞれの特徴を比べてみましょう。

	WSL2	仮想マシン	ネイティブ Linux
導入のしやすさ	◎	△	×
Windows との共存	◎	◎	×
動作の軽さ	◎	△	◎
Linux との互換性	○	◎	◎
GUI アプリの利用	△	◎	◎
初学者向き	◎	△	×

WSL2 Windows 上で Linux を動かす。導入が簡単で軽い。授業・研究用途に最適。

仮想マシン (VM) VirtualBox や VMware を使って PC の中に仮想的な PC を作る。自

由度は高いが、セットアップが複雑でメモリ消費が多い。

ネイティブ Linux PC に直接 Linux をインストールする（またはデュアルブート）。最も自由度が高いが、PC の設定を大きく変更する必要がある。

1.5 このマニュアルで使う Linux ディストリビューション

Linux にはいくつかの「ディストリビューション（distribution、略してディストロ）」があります。ディストリビューションとは、Linux の基本部分に様々なソフトウェアを組み合わせたパッケージのことです。

このマニュアルでは **Ubuntu 22.04 LTS** を使います。

- **Ubuntu**：最もユーザー数が多く、情報が豊富
- **22.04 LTS**：「Long Term Support」の略。長期間（5 年間）セキュリティアップデートが提供される安定版
- WSL でのデフォルトディストリビューションが Ubuntu のため、マニュアルや情報が充実

第 2 環境構築の全体像

2.1 これから何をするか

WSL と Python 環境を構築するために、大きく次のステップを踏みます。一度でわからなくても大丈夫です。各ステップで何をしているか概要を把握してから、次の章で詳しく進めましょう。

ステップ	内容	何をするか
1	WSL の有効化	Windows の設定を変えて WSL を使えるようにする
2	Ubuntu のインストール	Linux (Ubuntu) をインストールする
3	初期設定	ユーザー名・パスワードを設定し、基本的なアップデートをする
4	ターミナル操作の習得	画面の見方、入力方法、便利なショートカットを覚える
5	Linux の基本コマンドを覚える	ファイル操作などの基本コマンドを練習する
6	Python のインストール確認	Python が使える状態か確認する
7	仮想環境の構築	<code>venv</code> を使ってプロジェクト専用の Python 環境を作る
8	VS Code との連携 (任意)	エディタと WSL を連携させる

2.2 所要時間の目安

内容	所要時間（目安）	注意
WSL のインストール	15～30 分	ダウンロード速度による
ターミナル操作の練習	30～60 分	繰り返し練習することで定着
Linux 基本コマンドの習得	1～2 時間	一度で全部覚えなくて OK
Python・仮想環境の設定	15～30 分	

2.3 つまづきやすいポイント

環境構築は一見難しく見えますが、よくつまづく箇所は決まっています。あらかじめ知っておくと、焦らずに対処できます。

- パスワード入力時に文字が表示されない：セキュリティのためなので正常です（3.3.3 節 参照）
- コマンドを入力したが何も起きない：`Enter` を押し忘れていることが多いです
- エラーメッセージが出た：まずエラーの内容を読みましょう（第 16 章 参照）
- Windows の操作と Linux の操作を混同する：どちらのターミナルで作業しているか常に確認しましょう

メモ

このマニュアルを進めていて、うまくいかないことがあっても大丈夫です。エラーメッセージをよく読み、第 16 章（よくある質問とトラブルシューティング）を参照してください。

2.4 Windows と Linux の「並走」イメージ

WSL を使うと、Windows と Linux が同時に動いている状態になります。ファイルはそれぞれのシステムに存在しますが、お互いのファイルにアクセスすることもできます。

	Windows 側	Linux (Ubuntu) 側
ターミナル	PowerShell / コマンドプロンプト	Bash (Ubuntu ターミナル)
ホームフォルダ	C:\Users\ ユーザー名	/home/ ユーザー名
Python	Windows 用 Python	Ubuntu 用 Python3
パッケージ管理	winget / pip	apt / pip3

ポイント

作業は基本的に **Linux 側 (Ubuntu)** で行います。Windows 側のファイルに Linux から直接アクセスすることもできますが、Python の開発作業はすべて Linux 側のホームディレクトリ (**/home/ユーザー名**) で行うことを強く推奨します (詳細は第 14 章を参照)。

第3 WSL の導入

3.1 インストール前の確認

3.1.1 Windows のバージョン確認

WSL2 を使うには、Windows 10 バージョン 2004 (ビルド 19041) 以降、または Windows 11 が必要です。

バージョンを確認するには、**Win** キーを押して「winver」と入力し、**Enter** を押します。表示されたウィンドウで「バージョン」と「OS ビルド」を確認してください。

バージョンが要件を満たしていない場合は、Windows Update で最新版に更新してください。

3.1.2 仮想化機能の確認

WSL2 には CPU の仮想化機能が必要です。多くの最近の PC では有効になっていますが、まれに BIOS/UEFI で無効になっている場合があります。

タスクマネージャー (**Ctrl**+**Shift**+**Esc**) を開き、「パフォーマンス」タブの「CPU」画面で「仮想化：有効」となっていれば問題ありません。

3.2 WSL のインストール

3.2.1 PowerShell を管理者権限で起動する

スタートメニューで「PowerShell」と検索し、「管理者として実行」をクリックします。「このアプリがデバイスに変更を加えることを許可しますか？」と聞かれたら「はい」を選択します。

注意

「管理者として実行」は、PC の設定を変更する権限を持つモードです。このマニュアルでは WSL のインストール時のみ使用します。通常の作業では管理者権限は不要です。

3.2.2 WSL インストールコマンドの実行

PowerShell で以下のコマンドを入力し、**Enter** を押します。

```
> wsl --install
```

このコマンド 1 行で以下がすべて自動的行われます。

1. WSL に必要な Windows 機能の有効化
2. Linux カーネルのダウンロードとインストール
3. WSL2 をデフォルトに設定
4. Ubuntu のダウンロードとインストール

インストールには数分かかります（ネットワーク速度による）。完了すると、再起動を求めるメッセージが表示されます。

メモ**特定の Linux ディストリビューションを指定したい場合**

デフォルトでは Ubuntu がインストールされます。別のディストリビューションをインストールしたい場合は以下のコマンドで確認できます。

```
> wsl --list --online
```

指定してインストールする場合：

```
> wsl --install -d Ubuntu-22.04
```

3.2.3 PC を再起動する

インストールが完了したら、指示に従って PC を再起動します。

注意

再起動前に、作業中のファイルをすべて保存してください。

3.3 Ubuntu の初期設定

3.3.1 Ubuntu の初回起動

再起動後、スタートメニューから「Ubuntu」を検索して起動します（または、インストール完了後に自動で起動する場合があります）。

しばらく待つと、ユーザー名の入力を求めるメッセージが表示されます：

```
Enter new UNIX username:
```

3.3.2 ユーザー名の設定

Linux で使うユーザー名を入力します。以下の点に注意してください。

- 小文字のアルファベット・数字・ハイフンのみを使う（例：taro, student01）
- Windows のユーザー名と同じでも、違って OK
- スペースは使えない
- 後から変更が難しいため、シンプルなものを推奨

ユーザー名を入力して `Enter` を押します。

3.3.3 パスワードの設定

次にパスワードの入力を求められます。

```
New password:
```

パスワードを入力して `Enter` を押します。

注意

パスワードを入力しても画面に何も表示されません。これは正常です。

Linux のターミナルではセキュリティのために、パスワード入力中は文字も「●」も表示されません（エコーバックなし）。キーボードから入力はできているので、落ち着いて入力して `Enter` を押してください。確認のため同じパスワードをもう一度入力するよう求められます。

設定が完了すると、次のようなプロンプトが表示されます。

```
username@hostname:~$
```

このプロンプトが表示されたら、Ubuntu が使える状態です。

3.3.4 パッケージの更新

インストール直後は、パッケージ（ソフトウェア）のリストが古い状態です。以下のコマンドを実行してシステムを最新の状態にしましょう。

```
$ sudo apt update && sudo apt upgrade -y
```

メモ

コマンドの意味：

sudo 管理者（スーパーユーザー）権限でコマンドを実行する
apt Ubuntu のパッケージ管理ツール（ソフトを追加・削除するための仕組み）
update インストール可能なソフトウェアの一覧を最新化する
&& 左のコマンドが成功したら右のコマンドも実行する
upgrade 一覧に基づいてインストール済みのソフトを最新版に更新する
-y 途中で「本当にインストールしますか？」と聞かれたとき、自動的に「はい」と答える

初回は多くのパッケージが更新されるため、しばらく時間がかかる場合があります。

注意

sudo を使うとパスワードを求められることがあります。設定したパスワードを入力してください（画面には表示されません）。

3.4 Windows Terminal の設定（推奨）

Ubuntu のターミナルとして、**Windows Terminal** を使うことを強く推奨します。Windows Terminal は Microsoft が提供するモダンなターミナルアプリで、複数のタブを使えたり、見た目がカスタマイズできたりします。

3.4.1 Windows Terminal のインストール

Microsoft Store から「Windows Terminal」を検索してインストールします。Windows 11 には標準でインストールされています。

3.4.2 Ubuntu タブの開き方

Windows Terminal を起動し、タブバーの「+」ボタン横の「V」をクリックすると、使用できるシェルの一覧が表示されます。「Ubuntu」を選択すると Ubuntu のターミナルが開きます。

ポイント

Windows Terminal の設定 (`Ctrl`+`,`) で、Ubuntu をデフォルトのプロファイルに設定すると、起動時に自動で Ubuntu が開くようになります。

3.5 インストールの確認

以下のコマンドを実行して、正しくインストールされているか確認しましょう。

```
$ cat /etc/os-release
```

Ubuntu 22.04 などの表示が出れば成功です。

WSL のバージョンを Windows 側で確認することもできます。

```
> wsl --list --verbose
```

VERSION の列が 2 になっていれば WSL2 が使われています。

第 4 大学ネットワークのプロキシ設定

4.1 プロキシとは・なぜ必要か

大学のネットワークでは、セキュリティ管理のためにプロキシサーバーを経由してインターネットに接続することがあります。プロキシが設定されていないと、`apt install` や `pip install`、`git clone` などのコマンドでインターネットへの接続ができません。

メモ

本章のプロキシ設定は大学のネットワーク環境向けです。

プロキシアドレス：`http://proxy.cc.utsunomiya-u.ac.jp:8080`

自宅など大学外のネットワークでは設定が不要です（設定していると逆に繋がらなくなります）。他大学では異なるアドレスになります。

4.2 設定が必要な箇所の概要

コマンドの種類によってプロキシを読む設定ファイルが異なります。全部まとめて設定することを推奨します。

やりたいこと	設定ファイル	節
<code>pip</code> ・ <code>curl</code> ・ <code>python</code> など	<code>~/.bashrc</code>	4.3
システム全体・GUI アプリ	<code>/etc/environment</code>	4.4
<code>sudo apt install</code> など	<code>/etc/apt/apt.conf</code>	4.5
<code>git clone</code> ・ <code>git push</code> など	<code>~/.gitconfig</code>	4.6

ポイント

テキストエディタ vim の基本操作

この章では設定ファイルを編集するために vim を使います。

- ファイルを開く：vim **ファイル名**
- 編集開始：**i** (インサートモード)
- ファイル末尾に移動：**G** (大文字 G) → **o** (新しい行を追加して編集)
- 保存して終了：**Esc** → **:wq** → **Enter**
- 保存せずに終了：**Esc** → **:q!** → **Enter**

VS Code が使える環境では code **ファイル名** でそのまま開いても構いません。

4.3 ~/.bashrc：ターミナル全般のプロキシ設定

効果： ターミナルを開いたとき自動で読み込まれ、pip・curl・python などのコマンドに効きます。

```
$ vim ~/.bashrc
```

ファイルの**末尾** (**G** でジャンプ → **o** で新規行追加) に以下を追記します：

```
# ===== プロキシ設定 (大学ネットワーク用) =====  
export http_proxy="http://proxy.cc.utsunomiya-u.ac.jp:8080"  
export https_proxy="http://proxy.cc.utsunomiya-u.ac.jp:8080"  
export HTTP_PROXY="http://proxy.cc.utsunomiya-u.ac.jp:8080"  
export HTTPS_PROXY="http://proxy.cc.utsunomiya-u.ac.jp:8080"  
export no_proxy="localhost,127.0.0.1"  
export NO_PROXY="localhost,127.0.0.1"
```

Esc → **:wq** → **Enter** で保存して終了します。

保存後、設定を即時反映させます：

設定の即時反映と確認

```
$ source ~/.bashrc
```

```
$ echo $http_proxy  
http://proxy.cc.utsunomiya-u.ac.jp:8080
```

メモ

`source ~/.bashrc` は「このファイルを今すぐ読み込む」という命令です。通常は新しいターミナルを開いたときに自動で読み込まれますが、`source` を使うと再起動なしで今すぐ反映されます。

4.4 /etc/environment：システム全体のプロキシ設定

効果： ログイン時にすべてのユーザー・アプリに読み込まれます。`~/.bashrc` より広範囲に効きます。

```
$ sudo vim /etc/environment
```

ファイルの末尾に以下を追記します（このファイルでは `export` は不要です）：

```
http_proxy="http://proxy.cc.utsunomiya-u.ac.jp:8080"  
https_proxy="http://proxy.cc.utsunomiya-u.ac.jp:8080"
```

注意

このファイルでは `export` を付けないのが正しい書き方です。`/etc/environment` 独自の書式のため、`export` を付けると正しく読み込まれないことがあります。

保存後、設定はログアウト → ログインし直し（またはターミナルの再起動）で反映されます。

4.5 /etc/apt/apt.conf：apt 専用のプロキシ設定

効果： `sudo apt install` など `apt` コマンド専用のプロキシ設定です。`sudo` を使う `apt` コマンドは `~/.bashrc` の設定が効かないため、別途この設定が必要です。

```
$ sudo vim /etc/apt/apt.conf
```

以下を書き込みます（ファイルが新規作成の場合はそのまま入力）：

```
Acquire::http::proxy "http://proxy.cc.utsunomiya-u.ac.jp:8080";
Acquire::https::proxy "http://proxy.cc.utsunomiya-u.ac.jp:8080";
Acquire::ftp::proxy "http://proxy.cc.utsunomiya-u.ac.jp:8080";
```

注意

各行末尾のセミコロン「;」は必須です。忘れると apt がエラーになります。

メモ

/etc/apt/apt.conf の代わりに /etc/apt/apt.conf.d/proxy というファイルを新規作成して同じ内容を書いても同じ効果があります。設定ファイルを整理したい場合はこちらを使ってもよいです。

設定後、動作確認します：

```
$ sudo apt update
```

エラーなくパッケージリストが更新されれば成功です。

4.6 git のプロキシ設定

効果：git clone・git push などのネットワーク通信に効きます。

設定ファイルを直接編集せず、コマンドで設定します：

git プロキシの設定

```
$ git config --global http.proxy http://proxy.cc.utsunomiya-u.ac.jp:8080
```



```
$ git config --global https.proxy http://proxy.cc.utsunomiya-u.ac.jp:8080
```

設定確認：

```
$ git config --global --list | grep proxy
http.proxy=http://proxy.cc.utsunomiya-u.ac.jp:8080
https.proxy=http://proxy.cc.utsunomiya-u.ac.jp:8080
```

メモ

--global は「このユーザーのすべての Git リポジトリに適用する」という意味です。設定は ~/.gitconfig に自動的に書き込まれます。

第 5 ターミナルの基本操作

5.1 ターミナルとは何か

ターミナル（端末）とは、文字を入力してコンピュータを操作するためのインターフェースです。GUI（グラフィカルユーザーインターフェース）のようにマウスでアイコンをクリックするのではなく、コマンド（命令文）を文字で入力して操作します。

Linux の世界では、ターミナルのことを「コンソール」「シェル」「CLI」などとも呼びます。厳密には意味が異なりますが、日常会話ではほぼ同じ意味で使われます。

メモ

「シェル (shell)」とは？

シェルとは、ユーザーが入力したコマンドを受け取り、OS に伝えるプログラムです。「貝殻 (shell)」の名の通り、OS の核（カーネル）を覆う外側の部分という意味です。Ubuntu で標準的に使われるシェルは **Bash (Bourne Again SHell)** です。

5.2 プロンプトの読み方

Ubuntu のターミナルを開くと、最初に次のような文字列が表示されています。

```
username@hostname:~$
```

これを**プロンプト (prompt)**と言います。「次の入力を待っています」というサインです。各部分の意味は次の通りです。

部分	意味
username	現在ログインしているユーザー名（自分が設定した名前）
@	「～にいる」という区切り文字
hostname	コンピュータ（ホスト）の名前
:	区切り文字
~	現在いるディレクトリ（～はホームディレクトリを意味する。後述）
\$	「一般ユーザーとして入力を待っている」というサイン

メモ

\$ が # になっている場合は、管理者（root ユーザー）としてログインしています。通常の作業では \$ のまま使い、必要なときだけ `sudo` で一時的に管理者権限を取得します。

コマンドを入力するときは、\$ の後ろにカーソルが来ていることを確認してから入力します。入力後、`Enter` キーを押すとコマンドが実行されます。

5.3 よく使うショートカットキー

ターミナルではマウスよりキーボードで操作する場面が多いです。以下のショートカットを覚えておくと、作業が格段に楽になります。

5.3.1 コマンド実行・キャンセル

キー操作	動作
<code>Enter</code>	コマンドを実行する
<code>Ctrl</code> + <code>C</code>	実行中のコマンドを強制終了（キャンセル）する
<code>Ctrl</code> + <code>D</code>	ターミナルを終了する（または入力の終了を通知する）
<code>Ctrl</code> + <code>Z</code>	コマンドを一時停止する（バックグラウンドに移す）

注意

`Ctrl` + `C` は非常に重要なショートカットです。コマンドが動き続けて止まらないとき、予期しない動作をしているときは、`Ctrl` + `C` を押して強制終了してください。

5.3.2 カーソル移動

キー操作	動作
← / →	1 文字ずつカーソル移動
Ctrl + ← / Ctrl + →	単語単位でカーソル移動
Ctrl + A	行の先頭へ移動
Ctrl + E	行の末尾へ移動

5.3.3 履歴・補完

キー操作	動作
↑ / ↓	過去に実行したコマンドを履歴から呼び出す
Tab	コマンド名やファイル名を自動補完する
Tab Tab	補完候補が複数ある場合、一覧を表示する
Ctrl + R	コマンド履歴をキーワードで検索する

ポイント

Tab 補完を積極的に使いましょう！

ファイル名やコマンド名を途中まで入力して Tab を押すと、自動的に残りを補完してくれます。長いファイル名も、最初の数文字入力して Tab を押すだけで入力できます。タイプミスも減ります。Tab 補完は熟練者でも毎回使う重要な機能です。

5.3.4 画面操作

キー操作	動作
Ctrl + L	画面をクリアする（clear コマンドと同じ）
Shift + PageUp	画面を上スクロール
Shift + PageDown	画面を下スクロール

5.4 テキストのコピーとペースト

Windows Terminal では以下の方法でコピー・ペーストができます。

- コピー：テキストを選択すると自動的にコピーされる（またはマウス右クリック）
- ペースト：`Ctrl`+`V`、またはマウス右クリック

メモ

通常の Windows アプリでは `Ctrl`+`C` でコピーしますが、ターミナルでは `Ctrl`+`C` はコマンドのキャンセルです。コピーは選択するだけ（または右クリック）で行います。

5.5 コマンドの基本構造

ターミナルで実行するコマンドは、一般に次の構造をしています。

コマンド名 [オプション] [引数]

コマンド名 実行するプログラムの名前。例：`ls`, `cd`, `python3`

オプション コマンドの動作を変更する指示。- または -- で始まる。例：`-l`, `--help`

引数 コマンドの対象となる値（ファイル名など）。例：`/home/user`

例えば：

```
$ ls -l /home
```

この例では `ls` がコマンド名、`-l` がオプション（詳細表示）、`/home` が引数（表示対象のディレクトリ）です。

メモ

コマンド名、オプション、引数はすべて半角スペースで区切ります。全角スペースは使えません。また、大文字と小文字は区別されます。`ls` と `LS` は別のコマンドとして扱われます（`LS` というコマンドは存在しないのでエラーになります）。

第6 Unix/Linux の考え方に慣れる

6.1 ファイルシステムとは

コンピュータ上のファイルやフォルダは、**ファイルシステム**と呼ばれる仕組みで管理されています。ファイルシステムは木（ツリー）のような階層構造を持っています。

Windows でも Linux でも、ファイルシステムは「フォルダの中にフォルダやファイルがある」という階層構造は同じです。ただし、起点（ルート）の表し方やパス（場所を示す文字列）の書き方が異なります。

6.2 パスとは何か

パス (path) とは、ファイルやディレクトリの「住所」です。どこにそのファイルがあるかを文字列で表します。

	Windows	Linux (Ubuntu)
区切り文字	\ (バックスラッシュ)	/ (スラッシュ)
起点 (ルート)	C:\、D:\ など (ドライブ別)	/ (すべて 1 つのツリー)
ホームフォルダ	C:\Users\ ユーザー名	/home/ユーザー名
例	C:\Users\taro\Desktop	/home/taro/Desktop

6.3 重要なパス記号

Linux のパスには、特別な意味を持つ記号があります。これらを理解することがターミナル操作の基礎です。

6.3.1 / (ルートディレクトリ)

すべてのディレクトリ・ファイルの起点（最上位）です。Windows の「C: ドライブ」に相当しますが、Linux ではすべてのファイルが / 以下の 1 本のツリーに収まります。

```
$ ls /  
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  
run  sbin  srv  sys  tmp  usr  var
```

メモ

代表的なディレクトリの意味：

/home 一般ユーザーのホームディレクトリが集まる場所

/etc 設定ファイルが入っている場所

/usr ユーザーが使うプログラムやライブラリが入っている場所

/tmp 一時ファイルを置く場所（再起動すると消える）

/mnt Windows のドライブなど、外部のファイルシステムをマウント（接続）する場所

6.3.2 ~ (チルダ：ホームディレクトリ)

~ (チルダ) は自分のホームディレクトリを表す省略記号です。/home/ユーザー名 と同じ意味ですが、~ と書く方が短くて便利です。

```
# 以下の 2 つは同じ場所を表す（ユーザー名が taro の場合）  
$ cd ~  
$ cd /home/taro
```

ホームディレクトリは、自分専用の作業スペースです。ファイルやプロジェクトはここに作成していきます。

6.3.3 . (ドット：カレントディレクトリ)

. (ピリオド 1 つ) は現在いるディレクトリ（カレントディレクトリ）を表します。

```
# . は「今いる場所」を意味する
$ ls .          # 今いるディレクトリの内容を表示 (ls だけと同じ)
$ cp file.txt ./backup/ # 今いるディレクトリ内の backup フォルダに
                        コピー
```

メモ

. から始まるファイル名は隠しファイル (hidden file) です。例: `.bashrc`、`.gitignore`。通常の `ls` では表示されず、`ls -a` オプションで表示されます。

6.3.4 .. (ドット 2 つ: 親ディレクトリ)

.. (ピリオド 2 つ) は 1 つ上のディレクトリ (親ディレクトリ) を表します。

```
# 今 /home/taro/project にいるとする
$ cd ..          # → /home/taro に移動
$ cd ../../      # → /home に移動 (2 つ上に移動)
$ ls ..          # 1 つ上のディレクトリの内容を表示
```

6.4 絶対パスと相対パス

パスの書き方には 2 種類あります。

6.4.1 絶対パス (absolute path)

/ から始まる、ルートからの完全なパスです。どこにいても、その場所を正確に指定できます。

```
$ cd /home/taro/Documents # どこにいても Documents に移動できる
$ cat /etc/os-release      # OS の情報を表示
```

6.4.2 相対パス (relative path)

現在いる場所 (カレントディレクトリ) を基準にしたパスです。/ で始まりません。


```
# 今 /home/taro にいるとする
$ cd Documents           # → /home/taro/Documents に移動
$ cd ../project/src      # → /home/taro/project/src に移動
$ cd ../other_user       # → /home/other_user に移動
```

ポイント

どちらを使うべき？

- よく使う場所に移動：相対パスが便利（短く書ける）
- 別の場所から特定のファイルを指定：絶対パスが確実
- スクリプトの中で使う：絶対パスか ~ を使う（実行場所によらず正しく動く）

6.5 大文字・小文字の区別

Linux のファイルシステムでは、大文字と小文字は別のものとして扱われます（case-sensitive）。

```
$ touch File.txt file.txt FILE.txt # 3 つ別々のファイルとして作成される
$ ls
File.txt  FILE.txt  file.txt
```

Windows では File.txt と file.txt は同じファイルとして扱われますが、Linux では異なります。ファイル名を入力するときは大文字・小文字に注意してください。

6.6 Windows のファイルへ WSL からアクセスする

WSL では /mnt/c/ 以下に Windows の C: ドライブがマウント（接続）されています。

Windows のパス	WSL からのパス
C:\Users\taro	/mnt/c/Users/taro
D:\data\	/mnt/d/data/
C:\Users\taro\Desktop	/mnt/c/Users/taro/Desktop

注意

Python の作業は Linux 側（/home/）で行うことを強く推奨します。

/mnt/c/ 以下（Windows 側）にあるファイルに対して Python を実行すると、ファイルアクセスが遅くなる場合があります。また、パーミッション（権限）の扱いが異なり、予期しないエラーが起きることがあります。授業の課題ファイルは Linux 側の ~/ 以下にコピーして作業してください。

6.7 ファイルとディレクトリ

Linux では、「フォルダ」のことを**ディレクトリ（directory）**と呼びます。意味はほぼ同じですが、コマンドラインや技術的な文書では「ディレクトリ」という言葉を使います。

すべてのものが「ファイル」として管理されている（デバイスもディレクトリも）のが Linux の特徴ですが、初めのうちは「ディレクトリ = フォルダ」「ファイル = 普通のファイル」という理解で問題ありません。

6.8 パーミッション（権限）の概念

Linux では、ファイルやディレクトリに**パーミッション（permission、アクセス権）**があります。誰が読める（read）、書ける（write）、実行できる（execute）かを細かく設定できます。

```
$ ls -l
-rw-r--r-- 1 taro taro 1234 Apr 21 10:00 note.txt
drwxr-xr-x 2 taro taro 4096 Apr 21 10:00 project/
```

先頭の文字列（-rw-r--r-- など）がパーミッションを表します。初めのうちはあまり深く気にしなくて大丈夫ですが、「読み取り専用」「実行できない」などのエラーが出たときに確認する参考にしてください。

第 7 基本コマンドを使ってみよう

この章では、日常的なファイル操作で使う基本コマンドを 1 つずつ学びます。各コマンドについて、**名前の由来・何をするか・基本的な使い方・注意点**を説明します。

7.1 pwd — 今いる場所を確認する

由来：Print Working Directory（現在の作業ディレクトリを表示する）

何をするか：今自分がいるディレクトリ（場所）の絶対パスを表示します。「今どこにいるか」を確認するための最初のコマンドです。

```
$ pwd
/home/taro
```

オプションはほとんど使いません。迷ったらまず pwd で自分の位置を確認する習慣をつけましょう。

ポイント

ターミナルを開いた直後は、**ホームディレクトリ（/home/ユーザー名）**にいます。作業が進むにつれてディレクトリを移動していくので、「今どこにいるか」を意識することが大切です。

7.2 ls — ディレクトリの内容を表示する

由来：LiSt（一覧表示する）

何をするか：ディレクトリの中にあるファイルやディレクトリの一覧を表示します。Windows の「エクスプローラー」で中身を見ることに相当します。

```
$ ls          # 現在のディレクトリの内容を表示
$ ls /home    # /home ディレクトリの内容を表示
$ ls ~        # ホームディレクトリの内容を表示
```

7.2.1 よく使うオプション

オプション	説明
-l	詳細情報を縦に表示（パーミッション、サイズ、更新日時など）
-a	隠しファイル（. で始まるファイル）も表示する
-h	ファイルサイズを人間が読みやすい単位（KB, MB）で表示（-l と組み合わせる）
-la または -al	-l と -a を同時に指定（よく使う）

```
$ ls -l
total 24
drwxr-xr-x 2 taro taro 4096 Apr 21 10:00 Documents
drwxr-xr-x 2 taro taro 4096 Apr 21 09:30 project
-rw-r--r-- 1 taro taro  512 Apr 21 10:05 note.txt

$ ls -la
total 36
drwxr-xr-x  4 taro taro 4096 Apr 21 10:00 .
drwxr-xr-x 12 root root 4096 Apr 20 08:00 ..
-rw-r--r--  1 taro taro  220 Apr 20 08:00 .bash_logout
-rw-r--r--  1 taro taro 3526 Apr 20 08:00 .bashrc
drwxr-xr-x  2 taro taro 4096 Apr 21 10:00 Documents
```

メモ

`ls -la` の出力で表示される `.` と `..` は、「現在のディレクトリ自身」と「親ディレクトリ」を表しています（第 6 章 参照）。

7.3 cd — ディレクトリを移動する

由来：Change Directory（ディレクトリを変更する）

何をするか： 現在いる場所（カレントディレクトリ）を別のディレクトリに移動します。
Windows のエクスプローラーでフォルダをダブルクリックして開くことに相当します。

```
$ cd Documents          # Documents フォルダへ移動（相対パス）
$ cd /home/taro         # 絶対パスで移動
$ cd ~                  # ホームディレクトリへ移動
$ cd ..                 # 1 つ上（親ディレクトリ）へ移動
$ cd ../..              # 2 つ上へ移動
$ cd -                  # 直前にいたディレクトリへ戻る（便利！）
```

ポイント

cd だけ（引数なし）で入力すると、ホームディレクトリに戻ります。
迷子になったときはとりあえず cd を実行してみましょう。

注意

ディレクトリ名の大文字・小文字は区別されます。cd Documents と cd documents は異なります（存在しなければエラーになります）。Tab 補完を使えばタイプミスを防げます。

7.4 mkdir — ディレクトリを作成する

由来：MaKe DIRectory（ディレクトリを作る）

何をするか： 新しいディレクトリ（フォルダ）を作成します。

```
$ mkdir my_project      # my_project という名前のディレクトリを
                        作成
$ mkdir ~/workspace     # ホームに workspace を作成
$ mkdir -p project/src/data # 階層的に一気に作成（p = ） parents
```

オプション	説明
-------	----

-p	中間のディレクトリが存在しなくても、まとめて作成する。すでに存在していてもエラーにならない
----	---

メモ

ディレクトリ名の付け方のコツ：

- スペースの代わりに `_` (アンダースコア) や `-` (ハイフン) を使う
- 英数字・`_`・`-` のみを推奨（日本語は避ける）
- 小文字を使う慣習がある
- 例：`lecture01`, `machine_learning`, `2026_spring`

7.5 touch — 空のファイルを作成する

由来：**touch**（触れる＝タイムスタンプを更新する）

何をするか： 空のファイルを新規作成します。すでにファイルが存在する場合は、そのファイルの最終更新日時を現在時刻に更新します（内容は変更しない）。

```
$ touch note.txt           # note.txt という空のファイルを作成
$ touch script.py         # Python ファイルを作成（中身は空）
$ touch file1.txt file2.txt # 複数ファイルを一度に作成
```

メモ

`touch` はファイルを作るだけです。内容を書き込むには、テキストエディタ（`vim` など）や `VS Code` を使います。

7.6 cp — ファイルやディレクトリをコピーする

由来：**CoPy**（コピーする）

何をするか： ファイルやディレクトリを別の場所にコピーします。

基本的な使い方

```
$ cp note.txt backup.txt      # note.txt を backup.txt としてコピー
```

```
$ cp note.txt Documents/           # note.txt を Documents フォルダにコピー
$ cp -r project/ project_backup/    # ディレクトリを再帰的にコピー
```

オプション 説明

<code>-r</code>	ディレクトリを中身ごと再帰的にコピーする (recursive)。ディレクトリをコピーするときは必須
<code>-i</code>	コピー先にファイルが存在する場合、上書きする前に確認する (interactive)

注意

コピー先に同名のファイルが既に存在する場合、**確認なしに上書き**されます。大事なファイルを上書きしないよう注意してください。不安な場合は `-i` オプションを使いましょう。

7.7 mv — ファイルを移動・名前変更する

由来：MoVe（移動する）

何をするか： ファイルを別の場所に移動します。また、同じ場所で名前を変えることもできます（名前の変更 = ファイルを別名で「移動」する）。

```
$ mv old_name.txt new_name.txt      # ファイル名を変更する
$ mv note.txt Documents/           # Documents フォルダへ移動する
$ mv project/ ~/workspace/         # ディレクトリを移動する
$ mv *.txt archive/                # カレントディレクトリの .txt ファイルをすべて移動
```

メモ

`cp` と違い、`mv` は元のファイルが残りません。移動後は元の場所にファイルはなくなります。

7.8 rm — ファイルやディレクトリを削除する

由来：ReMove（削除する）

何をするか： ファイルやディレクトリを削除します。

```
$ rm note.txt           # ファイルを削除
$ rm -i file.txt        # 削除前に確認する (安全)
$ rm -r old_project/    # ディレクトリを中身ごと削除 () recursive
```

危険：rm は取り消しません

rm で削除したファイルはごみ箱に入りません。完全に消えます。
一度実行すると元に戻す方法はありません（特別な復元ツールを使わない限り）。

- 削除前に本当に不要かよく確認してください
- 不安なときは `-i` オプション（削除前に確認）をつけてください
- `rm -rf /` は絶対に実行しないでください（システム全体が消えます）

オプション 説明

<code>-i</code>	削除前に「本当に削除しますか？」と確認する (interactive)。安全のためおすすめ
<code>-r</code>	ディレクトリを中身ごと再帰的に削除する (recursive)。ディレクトリを削除するときは必須
<code>-f</code>	確認なしに強制削除する (force)。慎重に使うこと

7.9 cat — ファイルの内容を表示する

由来：conCATenate（連結する＝複数のファイルを結合して出力する）

何をするか： ファイルの内容をターミナルに表示します。複数ファイルを指定すると連結して表示します。

```
$ cat note.txt           # note.txt の内容を表示
$ cat file1.txt file2.txt # 2 つのファイルを連結して表示
$ cat /etc/os-release    # OS の情報ファイルを表示
```

メモ

大きいファイルの表示には `less` を使いましょう。
長いファイルを `cat` で表示すると、画面がスクロールされて最後の部分しか見えま

せん。ページ送りしながら読みたい場合は `less ファイル名` を使います（付録 B 参照）。

7.10 まとめ：基本コマンド早見表

コマンド	由来	用途
<code>pwd</code>	Print Working Directory	現在の場所を確認
<code>ls</code>	LiSt	ディレクトリの内容一覧
<code>cd</code>	Change Directory	ディレクトリ移動
<code>mkdir</code>	MaKe DIRectory	ディレクトリ作成
<code>touch</code>	touch	空ファイル作成
<code>cp</code>	CoPy	ファイル・ディレクトリのコピー
<code>mv</code>	MoVe	ファイルの移動・名前変更
<code>rm</code>	ReMove	ファイル・ディレクトリの削除（注意）
<code>cat</code>	conCATenate	ファイル内容の表示

第 8 コマンドの見方と覚え方

8.1 コマンドのヘルプを読む

コマンドの使い方を忘れたとき、インターネットで調べる前にターミナルから直接ヘルプを確認できます。

8.1.1 --help：簡易ヘルプを表示する

ほとんどのコマンドは `--help` オプションに対応しています。

```
$ ls --help      # ls のヘルプを表示
$ cp --help      # cp のヘルプを表示
$ python3 --help # python3 のヘルプを表示
```

オプションの一覧と簡単な説明が表示されます。最初は量が多くて圧倒されますが、「-l」などよく使うオプションの説明だけ確認する使い方です。

8.1.2 man：詳細マニュアルを表示する

由来：MANual（マニュアル）

`man` コマンドは、コマンドの完全な説明書（マニュアルページ）を表示します。

```
$ man ls      # ls のマニュアルを表示
$ man cp      # cp のマニュアルを表示
$ man man     # man 自身のマニュアルを表示
```

`man` の画面の操作方法：

キー	操作
↑ / ↓	1 行ずつスクロール
Space / f	次のページへ
b	前のページへ
/ + 検索語 + Enter	テキスト検索
n	次の検索結果へ
q	マニュアルを閉じる

メモ

man が表示している画面は「less」という閲覧ツールで動いています。q で閉じることを忘れずに。

8.2 補足コマンド集

8.2.1 whoami：ログインしているユーザー名を表示する

由来：WHO AM I（私は誰？）

```
$ whoami
taro
```

複数のユーザーがいる環境や、管理者権限で操作しているときに「今どのユーザーとして実行しているか」を確認するのに便利です。

8.2.2 clear：画面をクリアする

```
$ clear
```

ターミナルの画面をきれいに消去します。Ctrl+L でも同じことができます。以前の出力が多くなって画面が見づらくなったときに使います。

8.2.3 history：コマンド履歴を表示する

```
$ history          # 過去に実行したコマンドを一覧表示
$ history 20       # 直近 20 件だけ表示
$ history | grep python # 履歴から python を含むものを検索
```

ポイント

 キーで 1 つずつ履歴を遡れますが、+ を押すとキーワードで履歴を検索できます。py と入力すると最後に実行した python3 などが見つかります。

8.2.4 which：コマンドの実体がどこにあるか確認する

由来：WHICH（どれ？）

```
$ which python3
/usr/bin/python3
$ which ls
/usr/bin/ls
```

コマンドを実行したとき、実際にどのプログラムが動くかを確認できます。仮想環境を使っているとき、どの Python が使われているか確認するのに便利です。

```
$ which python3
/usr/bin/python3          # 仮想環境なし：システム Python

$ source .venv/bin/activate
(.venv) $ which python3
/home/taro/project/.venv/bin/python3 # 仮想環境の Python が使われている
```

8.2.5 exit：ターミナルを終了する

```
$ exit
```

ターミナルセッションを終了します。`Ctrl`+`D` でも同じです。

8.3 発展コマンド

8.3.1 less：ファイルをページ送りしながら見る

```
$ less large_file.txt      # ページ送りしながら内容を表示
```

`cat` と違い、画面 1 ページずつ表示するため大きいファイルも読みやすいです。`man` と同じキー操作 (`Space` で次のページ、`q` で終了) で操作します。

8.3.2 head / tail：ファイルの先頭・末尾を表示する

```
$ head file.txt           # 先頭 10 行を表示 (デフォルト)
$ head -n 20 file.txt     # 先頭 20 行を表示
$ tail file.txt           # 末尾 10 行を表示
$ tail -n 5 file.txt      # 末尾 5 行を表示
$ tail -f log.txt         # ファイルの末尾を監視し続ける (ログ確認に便利)
```

8.3.3 wc：行数・単語数・文字数を数える

由来：Word Count (単語数を数える)

```
$ wc file.txt             # 行数、単語数、文字数を表示
$ wc -l file.txt          # 行数のみ表示 (l = ) lines
$ wc -w file.txt          # 単語数のみ表示 (w = ) words
```

8.4 コマンドの構造を理解する

良いコマンドの使い手になるには、コマンドの構造パターンを理解しておく助けになります。

パターン	例	意味
コマンド	<code>pwd</code>	そのまま実行
コマンド 引数	<code>cd Documents</code>	引数（対象）を指定
コマンド -オプション	<code>ls -l</code>	動作を変更
コマンド --長いオプション	<code>ls --all</code>	長い形式のオプション
コマンド -o 引数	<code>head -n 5 file</code>	オプションと引数の組み合わせ
コマンド 1 コマンド 2	<code>history grep py</code>	パイプ（出力を次のコマンドへ）

メモ

短いオプションと長いオプション：

多くのコマンドは同じ機能に対して短い形式（`-a`）と長い形式（`--all`）の両方があります。短い形式は入力が速く、長い形式は意味がわかりやすいです。どちらを使っても結果は同じです。

8.5 コマンドを覚えるコツ

コマンドは一度で全部覚える必要はありません。以下の方針で少しずつ習得しましょう。

1. まず `pwd`, `ls`, `cd` の 3 つをマスターする
2. 次に `mkdir`, `touch`, `cat` を覚える
3. 必要になったとき `--help` や `man` で調べながら使う
4. 繰り返し使うことで自然に記憶に定着する

ポイント

Tab 補完を常に活用してください！

コマンド名やファイル名の途中まで入力して `Tab` を押すと自動補完されます。例：
`Doc` と入力して `Tab` → `Documents` に補完されます。タイプミスを防ぎ、時間も節約できます。

第9 練習用ディレクトリを作って操作してみよう

これまで学んだコマンドを組み合わせて、実際にファイル操作を練習しましょう。手を動かして繰り返すことで、コマンドが身につきます。

9.1 練習の前に：現在位置の確認

まずは現在地を確認します。

```
$ pwd
/home/taro
$ ls
```

ホームディレクトリにいることを確認してから始めましょう。別の場所にいる場合は `cd` でホームに戻ります。

```
$ cd ~
$ pwd
/home/taro
```

9.2 練習 1：ディレクトリ構造を作る

`practice` という練習用ディレクトリを作り、その中にサブディレクトリを作ります。

ステップ 1：練習用ディレクトリを作成する

```
$ mkdir practice
$ cd practice
$ pwd
```

```
/home/taro/practice
```

```
$ mkdir docs src data
$ ls
data  docs  src
```

```
$ mkdir -p src/utils src/models
$ ls src/
models  utils
```

最終的に以下のようなディレクトリ構造ができます。

```
$ find . -type d
.
./data
./docs
./src
./src/models
./src/utils
```

9.3 練習 2：ファイルを作成・確認する

```
$ touch docs/README.txt
$ touch src/main.py
$ touch data/sample.csv
$ ls docs/
README.txt
$ ls src/
main.py  models  utils
```

ファイルの内容を確認してみます（今は空なので何も表示されません）。


```
$ cat docs/README.txt何も表示されない  
( →空のファイル )
```

9.4 練習 3：ファイルのコピーと移動

```
# docs/README.txt を docs/README_backup.txt としてコピー  
$ cp docs/README.txt docs/README_backup.txt  
$ ls docs/  
README.txt README_backup.txt
```

```
# mv コマンドで名前を変更する  
$ mv docs/README_backup.txt docs/OLD_README.txt  
$ ls docs/  
OLD_README.txt README.txt
```

```
# OLD_README.txt を data フォルダへ移動する  
$ mv docs/OLD_README.txt data/  
$ ls docs/  
README.txt  
$ ls data/  
OLD_README.txt sample.csv
```

9.5 練習 4：ディレクトリをまるごとコピー

Ubuntu/WSL ターミナル

```
# practice 全体を backup として別のディレクトリにコピー  
$ cd ~  
$ cp -r practice practice_backup  
$ ls  
practice practice_backup
```

```
$ ls practice_backup/  
data docs src
```

9.6 練習 5：不要ファイルの削除

```
$ cd ~/practice  
$ rm -i data/OLD_README.txt      # -i で確認しながら削除  
rm: remove regular empty file 'data/OLD_README.txt'? y  
$ ls data/  
sample.csv
```

```
$ cd ~  
$ rm -r practice_backup          # ディレクトリごと削除  
$ ls  
practice
```

危険

`rm -r` は指定したディレクトリの中身をすべて削除します。確認なしに実行されるため、対象を間違えないように注意してください。心配な場合は `ls` で削除対象を確認してから実行しましょう。

9.7 まとめ演習

以下のディレクトリ構造を **コマンドのみ**で作ってみましょう。

```
~/lecture/  
├── week01/  
│   ├── notes.txt  
│   └── exercise.py  
├── week02/  
│   └── notes.txt  
└── resources/  
    └── textbook.txt
```

メモ

ヒント：

1. `mkdir -p` でまとめてディレクトリを作れます
2. `touch` で空のファイルを作れます
3. 最後に `ls -R /lecture/` で構造を確認しましょう

できたら以下で確認してください。

```
$ ls -R ~/lecture/
```

第 10 Python を使う準備

10.1 Ubuntu と Python

Ubuntu (Linux) には、多くの場合 **Python 3** が最初からインストールされています。ただし、授業で使うパッケージを追加したり、仮想環境を作ったりするためにいくつかの準備が必要です。

メモ

Ubuntu では `python3` というコマンドを使います。`python` (数字なし) は Ubuntu によっては存在しないか、Python 2 を指すことがあります。このマニュアルでは **必ず** `python3` を使います。

10.2 Python のインストール確認

```
$ python3 --version
Python 3.10.12
```

「Python 3.x.x」と表示されれば OK です。

コマンドが見つからない場合 (`command not found` と表示された場合) は、次節の手順でインストールします。

10.3 Python のインストール

```
$ sudo apt update
$ sudo apt install -y python3 python3-pip python3-venv
```

メモ

インストールするパッケージの意味：

python3 Python 3 本体

python3-pip Python のパッケージ管理ツール (pip3 コマンド)

python3-venv 仮想環境を作るためのモジュール (venv ← 次章で使います)

インストール後、再確認します。

```
$ python3 --version
Python 3.10.12
$ pip3 --version
pip 22.0.2 from /usr/lib/python3/dist-packages/pip (python
  3.10)
```

10.4 pip とは何か

pip (Python Package Installer) は、Python のパッケージ (ライブラリ) をインターネットからインストールするためのツールです。

PyPI (Python Package Index) というサーバーに何十万種類ものパッケージが登録されており、**pip** を使って簡単にインストールできます。

```
$ pip3 install numpy           # numpy をインストール
$ pip3 install numpy pandas    # 複数まとめてインストール
$ pip3 list                    # インストール済みパッケージの一覧
$ pip3 show numpy              # numpy の情報を表示
$ pip3 uninstall numpy         # numpy をアンインストール
```

注意

この段階でグローバルに `pip3 install` するのは避けましょう。
`sudo pip3 install` でシステム全体にパッケージをインストールすることは**推奨しません**。なぜ危険なのかは次の章で説明します。授業で使うパッケージは、次章で説明する**仮想環境**の中でインストールします。

10.5 Python の動作確認

Python が正しく動くか確認してみましょう。

```
$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0]
Type "help", "copyright", "credits" or "license" for more
information.
>>>
```

`>>>` が表示されたら、Python のインタラクティブモード (REPL) が起動しています。試しに計算してみましょう。

```
>>> 1 + 2
3
>>> print("Hello, WSL!")
Hello, WSL!
>>> import sys
>>> sys.version
'3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0]'
>>> exit()
$
```

メモ

インタラクティブモードを終了するには `exit()` と入力するか、`Ctrl`+`D` を押します。

10.6 Python スクリプトファイルの実行

Python コードをファイルに書いて実行してみましょう。

```
$ cd ~  
$ touch hello.py
```

vim でファイルを編集します（VS Code が使える場合は code hello.py でも OK）。

```
$ vim hello.py
```

vim が開いたら **i** キーを押してインサートモードに入り、以下を入力します：

Listing 10.1 hello.py

```
print("Hello , WSL!")  
print("Python が動いています。")
```

入力が終わったら **Esc** を押し、:wq と入力して **Enter** で保存して終了します。

```
$ python3 hello.py  
Hello , WSL!  
Python が動いています。
```

これで Python スクリプトが実行できることを確認できました。

第 11 仮想環境とは何か

11.1 なぜ仮想環境が必要か

Python のパッケージ（ライブラリ）は、インストールする場所が重要です。何も考えずに `pip install` すると、すべてのパッケージが「システム全体」にインストールされます。これが将来、様々な問題を引き起こします。

11.1.1 依存関係の衝突の例

次のようなシナリオを想像してください。

	授業 A（機械学習）	授業 B（Web 開発）
必要なバージョン	numpy 1.21	numpy 1.24

同じ PC に授業 A と授業 B の環境を両方作ろうとすると、numpy は 1 つのバージョンしかインストールできないため、どちらかが正常に動かなくなります。

状況	結果
numpy 1.21 だけインストール	授業 B が動かない
numpy 1.24 だけインストール	授業 A が動かない
1.24 にアップグレード	授業 A が壊れる

これが**依存関係の衝突**です。大学の授業では複数の科目で Python を使うことも多く、それぞれ必要なライブラリのバージョンが異なることがよくあります。

11.1.2 グローバル環境汚染

`pip install` を繰り返すと、システム全体の Python 環境（グローバル環境）にどんどんパッケージが蓄積されます。

- 使わなくなったパッケージが残り続ける
- バージョンアップによって他のプロジェクトが突然壊れる
- 「なぜか動かなくなった」の原因が特定しにくくなる
- 他人の環境で同じコードが動かないことがある

11.1.3 環境の再現が難しくなる

研究や課題では、「同じ環境で同じ結果を再現できること」が重要です。グローバル環境では、どのパッケージがどのバージョンで入っているか管理が難しく、「自分の PC では動くが先生や TA の PC では動かない」という事態が起きやすいです。

11.2 仮想環境の仕組み

仮想環境 (virtual environment) とは、プロジェクトごとに独立した Python の箱を作る仕組みです。

それぞれの仮想環境は：

- 独自の Python インタープリタのコピー（またはシンボリックリンク）を持つ
- 独自のパッケージ置き場を持つ
- 他の環境のパッケージには影響しない

仮想環境のイメージ

システム全体

- └ Python 3.10 (本体)
 - └ 授業 A 用仮想環境 (.venv) : numpy 1.21, scikit-learn 1.0
 - └ 授業 B 用仮想環境 (.venv) : numpy 1.24, flask 2.3
 - └ 研究 用仮想環境 (.venv) : numpy 1.25, torch 2.0

各仮想環境は完全に独立しており、一方を変更しても他方には影響しません。

11.3 仮想環境を使うメリット

依存関係の分離 プロジェクトごとに必要なパッケージだけを入れられる

バージョン管理 授業 A は numpy 1.21、授業 B は numpy 1.24、というように並行管理できる

環境の再現性 requirements.txt に書き出せば他の PC でも同じ環境を再現できる

削除が簡単 不要になった環境はディレクトリを削除するだけで完全に消える

システムを汚さない Ubuntu のシステム Python は変更しないので、OS が安定する

ポイント

仮想環境は「プロジェクトフォルダの中」に作るのが基本です。

プロジェクトごとに `.venv` というフォルダを作り、その中に仮想環境を置きます。そうすることで、プロジェクトとその環境をセットで管理できます。

11.4 このマニュアルでの方針

本マニュアルでは、以下の方針で仮想環境を扱います。

ツール	位置づけ
venv (標準)	授業での標準環境。Python 標準ライブラリの機能で追加インストール不要
uv (発展)	より便利な代替ツール。慣れたら試してみよう (第 13 章)

メモ

授業や演習でのサポートは **venv** を基準とします。TA や授業担当者への質問も **venv** を使っていると答えやすくなります。uv を使う場合は自己責任で試してください。

第 12 venv を使った標準的な Python 環境構築

12.1 venv の基本的な流れ

仮想環境の使い方は次の 4 ステップです。

1. 作成：仮想環境ディレクトリを作る (`python3 -m venv`)
2. 有効化：仮想環境を有効にする (`source .venv/bin/activate`)
3. 使用：パッケージをインストールして作業する
4. 無効化：作業が終わったら仮想環境を出る (`deactivate`)

12.2 ステップ 1：プロジェクトディレクトリの作成

授業や課題ごとにフォルダを作り、その中で仮想環境を管理します。

```
$ mkdir ~/lecture01
$ cd ~/lecture01
$ pwd
/home/taro/lecture01
```

12.3 ステップ 2：仮想環境の作成

`python3 -m venv` コマンドで仮想環境を作ります。慣例として `.venv` という名前にすることが多いです。

```
$ python3 -m venv .venv
```

このコマンドを実行すると、`.venv` というディレクトリが作成されます。

```
$ ls -la
total 12
drwxr-xr-x  3 taro taro 4096 Apr 21 10:00 .
drwxr-xr-x 12 taro taro 4096 Apr 21 09:55 ..
drwxr-xr-x  5 taro taro 4096 Apr 21 10:00 .venv
```

メモ

コマンドの意味：`python3 -m venv .venv`

`python3` Python 3 を使う

`-m venv venv` モジュールを実行する（`-m` は「モジュールとして実行」の意味）

`.venv` 作成する仮想環境ディレクトリの名前（`.` から始まるので隠しディレクトリ）

12.4 ステップ 3：仮想環境の有効化

仮想環境を「有効化 (activate)」することで、この仮想環境内の Python と pip を使う状態になります。

```
$ source .venv/bin/activate
(.venv) $
```

有効化されると、プロンプトの先頭に `(.venv)` という表示が追加されます。これが仮想環境が有効になっているサインです。

メモ

`source` コマンドとは？

`source` は、指定したスクリプトを現在のシェルで実行するコマンドです。通常のコマンドは子プロセスで実行されますが、`source` を使うと現在のシェルに直接変更を加えられます。仮想環境の有効化では、環境変数（`PATH` など）を現在のシェルに設定する必要があるため、`source` を使います。

有効化後は `which python3` で使用している Python が仮想環境内のものであることを確認できます。

```
(.venv) $ which python3
/home/taro/lecture01/.venv/bin/python3
(.venv) $ which pip3
/home/taro/lecture01/.venv/bin/pip3
```

12.5 ステップ 4：パッケージのインストール

仮想環境が有効な状態で `pip3 install` を実行すると、この仮想環境の中だけにパッケージがインストールされます。

```
(.venv) $ pip3 install numpy
Collecting numpy
  Downloading numpy-1.26.4-cp310-cp310-linux_x86_64.whl (15.9 MB)
  ...
Successfully installed numpy-1.26.4

(.venv) $ pip3 install matplotlib pandas
```

インストール済みのパッケージを確認するには：

```
(.venv) $ pip3 list
Package           Version
-----
matplotlib        3.9.0
numpy              1.26.4
pandas             2.2.2
pip                22.0.2
setuptools         59.6.0
```

12.6 requirements.txt による環境の共有・再現

インストールしたパッケージのリストを `requirements.txt` に書き出しておくと、別の PC や別の環境で同じパッケージを一括インストールできます。

```
(.venv) $ pip3 freeze > requirements.txt
(.venv) $ cat requirements.txt
matplotlib==3.9.0
numpy==1.26.4
pandas==2.2.2
```

```
# 別の PC でこのプロジェクトを再現するとき
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip3 install -r requirements.txt
```

ポイント

requirements.txt は Git にコミットしましょう。
プロジェクトを GitHub などで管理する場合、requirements.txt をコミットしておく、チームメンバーや TA が同じ環境を簡単に再現できます。

12.7 ステップ 5：仮想環境の無効化

作業が終わったら仮想環境を無効化します。

```
(.venv) $ deactivate
$
```

プロンプトから (.venv) が消えれば、仮想環境が無効化されています。

メモ

deactivate はコマンドではなく、仮想環境を有効化したときに設定されたシェルの関数です。仮想環境が有効なときのみ使えます。

12.8 仮想環境の削除

不要になった仮想環境は、ディレクトリを削除するだけです。

```
$ rm -r .venv      # .venv ディレクトリを削除
```

再度使いたい場合は `python3 -m venv .venv` と `pip3 install -r requirements.txt` で復元できます。

12.9 トラブル：仮想環境を有効化したのに（.venv）が出ない

注意

`./venv/bin/activate` や `sh .venv/bin/activate` など、`source` を使わずに実行した場合は有効化されません。必ず `source .venv/bin/activate` の形式で実行してください。

また、仮想環境のパスが間違っていないか確認してください。

```
$ ls .venv/bin/
activate  activate.csh  activate.fish  pip  pip3  python3
...
```

12.10 作業フローのまとめ

毎回の作業手順をまとめます。

順序	コマンド	タイミング
1	<code>cd /プロジェクト名</code>	作業開始時：プロジェクトへ移動
2	<code>source .venv/bin/activate</code>	作業開始時：仮想環境を有効化
3	<code>pip3 install パッケージ名</code>	必要なときだけ
4	（作業する）	—
5	<code>deactivate</code>	作業終了時（任意）

第 13 オプション：uv を使った快適な環境構築

メモ

この章は発展的な内容（オプション）です。venv の使い方に慣れてから、よりよいツールを試したい方向けです。授業でのサポート対象は venv です。uv でわからないことが起きた場合は、まず venv に戻ることを検討してください。

13.1 uv とは何か

uv は、Astral 社が開発した Python パッケージ管理ツールです。pip や venv の代替として使えます。

- 高速 Rust で書かれており、pip より 10~100 倍速い
- シンプル venv + pip の操作をほぼ同じコマンドで統一
- 再現性 uv.lock ファイルで依存関係を厳密に管理
- 互換性 既存の requirements.txt もそのまま使える

操作	venv + pip	uv
仮想環境を作る	<code>python3 -m venv .venv</code>	<code>uv venv</code>
有効化	<code>source .venv/bin/activate</code>	<code>source .venv/bin/activate</code>
パッケージ追加	<code>pip3 install numpy</code>	<code>uv add numpy</code>
環境の同期	<code>pip3 install -r requirements.txt</code>	<code>uv sync</code>

13.2 uv のインストール

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

インストール後、ターミナルを再起動するか `source ~/.bashrc` を実行します。

```
$ uv --version
uv 0.4.x
```

メモ

`curl -LsSf ... | sh` は、インターネットからスクリプトをダウンロードして直接実行するコマンドです。信頼できるソース（ここでは Astral 社の公式サイト）からのみ実行してください。

13.3 uv で仮想環境を作る

```
$ mkdir ~/lecture02
$ cd ~/lecture02
$ uv venv
Using CPython 3.10.12
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
```

`source .venv/bin/activate` で有効化するのは `venv` と同じです。

```
$ source .venv/bin/activate
(.venv) $
```

13.4 uv でパッケージを追加する

```
(.venv) $ uv add numpy
Resolved 1 package in 23ms
Installed 1 package in 8ms
+ numpy==1.26.4

(.venv) $ uv add matplotlib pandas
```

uv add は pip3 install より高速で、依存関係の解決も自動で行います。

13.5 uv の依存管理ファイル

uv add を実行すると、pyproject.toml と uv.lock が自動生成されます。

- pyproject.toml：プロジェクトの設定と直接の依存パッケージを記述
- uv.lock：すべての依存関係を厳密に固定するロックファイル

```
(.venv) $ cat pyproject.toml
[project]
name = "lecture02"
version = "0.1.0"
dependencies = [
    "matplotlib>=3.9.0",
    "numpy>=1.26.4",
    "pandas>=2.2.2",
]
```

他の環境で同じ構成を再現するには：

```
$ uv sync      # uv.lock に基づいてパッケージをインストール
```

13.6 uv を使う場面と venv を使う場面

場面	推奨	理由
授業・演習の標準環境	venv	サポートが充実している
TA・教員への質問	venv	共通の認識を持ちやすい
自分の個人プロジェクト	uv	より快適・高速
大規模なパッケージ管理	uv	依存解決が強力
研究・開発	uv	再現性が高い

ポイント

venv と uv は共存できます。授業フォルダでは venv、自分のプロジェクトでは uv、
というように使い分けることも問題ありません。

第 14 推奨される作業スタイル

14.1 ディレクトリ構成の推奨

Linux 側のホームディレクトリ（~）の中で、授業・プロジェクトごとにフォルダを分けることを推奨します。

~/（ホームディレクトリ）

```

├── lecture/                ← 授業ごとのフォルダ
│   ├── machine_learning/
│   │   ├── .venv/         ← 仮想環境（このフォルダ専用）
│   │   ├── week01.py
│   │   ├── week02.py
│   │   └── requirements.txt
│   └── web_development/
│       ├── .venv/
│       ├── app.py
│       └── requirements.txt
├── research/              ← 研究用フォルダ
│   ├── project_a/
│   │   ├── .venv/
│   │   └── ...
└── sandbox/              ← お試し・実験用
    └── ...

```

ポイント

仮想環境は各プロジェクトフォルダの内側に置くのが基本です。そうすることで、プロジェクトと環境が一体になり管理しやすくなります。

14.2 毎回の作業フロー

毎回の作業は以下の流れが基本です。

```
$ cd ~/lecture/machine_learning      # プロジェクトへ移動
$ source .venv/bin/activate           # 仮想環境を有効化
(.venv) $ python3 week03.py           # 作業する
(.venv) $ deactivate                  # 終了
```

14.3 Windows 側ファイルとの使い分け

注意

Python の作業ファイルは Linux 側（~/）に置いてください。

Windows 側（/mnt/c/）でファイルを作成して Python を実行することは技術的には可能ですが、以下の理由から推奨しません。

- ファイルアクセスが遅くなる（特に小さいファイルが多いとき）
- パーミッション関連のエラーが起きることがある
- 改行コードの違いによる問題が起きることがある

例外： Windows 側で作成したデータファイル（画像、CSV など）を処理する場合は、Linux 側にコピーしてから使うか、パスで参照してください。

```
# Windows の Desktop にある data.csv を Linux 側のプロジェクトへコピー
$ cp /mnt/c/Users/taro/Desktop/data.csv ~/lecture/
  machine_learning/
```

14.4 .gitignore で仮想環境を除外する（発展）

Git でプロジェクトを管理する場合、.venv ディレクトリはコミットしないようにします。仮想環境は requirements.txt から再現できるため、Git に含める必要はなく、含めるとリポジトリが大きくなります。

```
$ echo ".venv/" >> .gitignore
$ echo "__pycache__/" >> .gitignore
$ echo "*.pyc" >> .gitignore
```

14.5 ターミナルを複数使う

作業が増えてきたら、Windows Terminal のタブを複数開いて使い分けましょう。

- タブ 1：授業 A の作業
- タブ 2：授業 B の作業
- タブ 3：ファイル操作・確認用

それぞれのタブで別の仮想環境を有効化した状態にできます。

第 15 VS Code と連携する

15.1 VS Code とは

Visual Studio Code (VS Code) は、Microsoft が無料で提供する高機能なテキストエディタです。Python の開発に非常によく使われており、コード補完、デバッグ、Git 連携など多彩な機能を持ちます。

WSL と VS Code を連携させることで、Windows の VS Code から WSL 内のファイルを快適に編集できます。

15.2 VS Code のインストール (Windows 側)

VS Code は Windows にインストールします (WSL 側ではありません)。

1. <https://code.visualstudio.com/> にアクセス
2. 「Download for Windows」 ボタンをクリック
3. ダウンロードしたインストーラーを実行
4. インストール中の「PATH に追加する」オプションは**有効**にする (デフォルトで ON)

15.3 WSL 拡張機能のインストール

VS Code の拡張機能「WSL」をインストールすることで、VS Code から WSL 内のファイルを直接編集できるようになります。

1. VS Code を起動する
2. 左サイドバーの拡張機能アイコン (`Ctrl` + `Shift` + `X`) をクリック
3. 検索ボックスに「WSL」と入力
4. 「WSL」(Microsoft 製) をインストール

15.4 WSL から VS Code を開く

WSL のターミナルで `code` コマンドを使って VS Code を起動できます。

```
$ cd ~/lecture/machine_learning
$ code .           # 「.」は現在のディレクトリを意味する。
```

初回は VS Code サーバーを WSL 側にインストールする処理が走ります（数分かかることがあります）。次回以降は素早く起動します。VS Code の左下隅に「WSL: Ubuntu」と表示されていれば、WSL との連携が成功しています。

ポイント

`code .` は WSL ターミナルから実行します。

Windows 側の PowerShell から実行した場合は Windows モードで開きます。WSL 内のファイルを効率よく編集するため、必ず WSL ターミナルから `code .` を実行してください。

15.5 Python 拡張機能のインストール

Python のコード補完やデバッグを使うために、「Python」拡張機能をインストールします。

1. WSL モードで VS Code が開いている状態で
2. 拡張機能マーケットプレイス (`Ctrl` + `Shift` + `X`) を開く
3. 「Python」(Microsoft 製)を検索してインストール

メモ

VS Code の拡張機能には「Windows 側にインストール」と「WSL 側にインストール」の区別があります。Python 拡張機能は WSL モードで開いているときにインストールすると WSL 側に入ります。

15.6 仮想環境のインタープリタを選択する

仮想環境が有効化されている状態で VS Code を開くと自動的に認識されることが多いですが、手動で選択する方法も知っておきましょう。

1. `Ctrl` + `Shift` + `P` でコマンドパレットを開く
2. 「Python: Select Interpreter」と入力して選択

3. 一覧から `.venv` 内の Python（例：`.venv/bin/python3`）を選択

選択後、VS Code の右下に選択した Python のバージョンが表示されます。

15.7 基本的な VS Code の使い方

ショートカット	機能
<code>Ctrl</code> + <code>N</code>	新規ファイルを作成
<code>Ctrl</code> + <code>S</code>	保存
<code>Ctrl</code> + <code>'</code>	統合ターミナルを開く（WSL のターミナルが開く）
<code>Ctrl</code> + <code>Shift</code> + <code>P</code>	コマンドパレットを開く
<code>Ctrl</code> + <code>F</code>	ファイル内を検索
<code>Ctrl</code> + <code>/</code>	コメントアウト・解除
<code>Ctrl</code> + <code>Space</code>	コード補完候補を表示

ポイント

`Ctrl` + `'` で開く統合ターミナルは、そのまま WSL のターミナルとして使えます。VS Code の画面下部でファイル編集とターミナル操作を同時に行えるので大変便利です。

15.8 Python ファイルの実行（VS Code から）

VS Code で Python ファイルを開いたとき、右上の「**実行ボタン（三角アイコン）**」をクリックすると統合ターミナルで実行されます。または以下の方法でも実行できます。

- `Ctrl` + `F5`：デバッグなしで実行
- 統合ターミナルで `python3 ファイル名.py` を手動実行

第 16 よくある質問とトラブルシューティング

16.1 インストール・起動時のトラブル

16.1.1 wsl --install でエラーが出る

症状： wsl --install を実行してもエラーになる。

確認事項：

1. PowerShell を管理者として実行しているか確認
2. Windows のバージョンが要件を満たしているか確認 (winver で確認)
3. 「コントロールパネル → プログラム → Windows の機能の有効化または無効化」で「Windows Subsystem for Linux」と「仮想マシン プラットフォーム」が有効になっているか確認

解決策：

```
> dism.exe /online /enable-feature /featurename:Microsoft-  
Windows-Subsystem-Linux /all /norestart  
> dism.exe /online /enable-feature /featurename:  
VirtualMachinePlatform /all /norestart
```

上記を実行後、再起動してから再度 wsl --install を試してください。

16.1.2 Ubuntu が起動しない・「参照しているコンポーネントが見つからない」

解決策：

```
# WSL カーネルの更新
> wsl --update
> wsl --shutdown
> wsl
```

16.1.3 「0x80370102」エラー

BIOS/UEFI で仮想化 (Intel VT-x / AMD-V) が無効になっている可能性があります。PC メーカーのマニュアルを参照し、BIOS の仮想化設定を有効にしてください。

16.2 ターミナル操作のトラブル

16.2.1 パスワードを入力しても何も表示されない

これは正常な動作です。

Linux のターミナルでは、パスワード入力中はセキュリティのためカーソルの動きも含めて何も表示されません (エコーバックなし)。入力はできているので、落ち着いてパスワードを入力して **Enter** を押してください。

16.2.2 コマンドが終わらない・止まったように見える

Ctrl+**C** で強制終了してください。ほとんどの場合、これで元のプロンプトに戻ります。

```
$ python3 some_script.py (終わらない場合)
...
^C                               # Ctrl+C を押すとう表示される
$
```

16.2.3 Tab 補完が効かない

bash を使っているか確認してください。echo \$SHELL を実行して /bin/bash と表示されれば Bash が使われています。

また、補完候補が複数ある場合は **Tab** を 2 回押すと一覧が表示されます。

16.3 コマンドのトラブル

16.3.1 「command not found」と出る

コマンドが見つからないというエラーです。以下を確認してください。

- コマンドのスペルが正しいか（大文字・小文字も含めて）
- コマンドがインストールされているか（`which` コマンド名 で確認）
- パスが通っているか

```
$ python      # エラーになることがある
-bash: python: command not found
$ python3     # こちらを使う
Python 3.10.12 ...
```

16.3.2 「Permission denied」と出る

アクセス権がない（パーミッション不足）というエラーです。

- ファイルを読もうとした場合：自分に読み取り権限がない。`ls -l` でパーミッションを確認
- システムファイルを変更しようとした場合：`sudo` が必要
- スクリプトを実行しようとした場合：`chmod +x` スクリプト名 で実行権限を付与

16.3.3 「No such file or directory」と出る

ファイルやディレクトリが見つからないエラーです。

- パスのスペルを確認する（大文字・小文字も）
- `pwd` で今いる場所を確認し、`ls` でファイルが存在するか確認する
- 相対パスが正しいか確認する

16.4 Python・pip のトラブル

16.4.1 「pip: command not found」または「pip3: command not found」

pip がインストールされていない可能性があります。

```
$ sudo apt update
$ sudo apt install -y python3-pip
```

16.4.2 pip install でエラーが出る (externally-managed-environment)

Ubuntu 23.04 以降では、システムの Python 環境への直接インストールが制限されている場合があります。

解決策： 仮想環境を有効化してからインストールしてください。

```
$ source .venv/bin/activate      # まず仮想環境を有効化
(.venv) $ pip3 install numpy     # 仮想環境内にインストール
```

16.4.3 インストールしたパッケージが見つからない (ImportError)

仮想環境が有効化されていない可能性があります。

```
$ which python3
/usr/bin/python3                # ←仮想環境が無効 (システムの)   Python

$ source .venv/bin/activate
(.venv) $ which python3
/home/taro/project/.venv/bin/python3  # ←仮想環境の   Python
```

16.4.4 仮想環境の有効化コマンドを入力したが (.venv) が表示されない

`source` を使っているか確認してください。

```
$ .venv/bin/activate            # ←これは動かない
$ sh .venv/bin/activate         # ←これも動かない
```

```
$ source .venv/bin/activate    # ← source を使う
(.venv) $
```

16.5 その他のトラブル

16.5.1 rm で削除したファイルを元に戻したい

残念ながら、基本的には戻せません。

Linux の `rm` はごみ箱に入らず、ファイルが直接削除されます。大事なファイルは事前にバックアップするか、削除前に `-i` オプションで確認するようにしましょう。

ポイント

Git を使ってプロジェクトを管理していれば、コミットした時点まで戻すことができます。大事なプロジェクトは Git で管理することを強く推奨します。

16.5.2 Windows 側のファイルに WSL からアクセスできない

Windows の C ドライブは `/mnt/c/` でアクセスできます。

```
$ ls /mnt/c/Users/
taro  Public  Default

$ ls /mnt/c/Users/taro/Desktop/
```

16.5.3 sudo apt update でエラーが出る

インターネット接続を確認してください。接続に問題がない場合、DNS の設定が原因のことがあります。

```
$ cat /etc/resolv.conf
```

`nameserver` の行が存在するか確認してください。ない場合や、アドレスが正しくない場合は WSL を再起動してみてください。

```
> wsl --shutdown  
> wsl
```

16.5.4 文字化けする・日本語が表示されない

Windows Terminal を使っていれば、通常は日本語も正しく表示されます。フォントを変更すると改善することがあります。Windows Terminal の設定 (`Ctrl`+`,`) で、プロファイルのフォントを「BIZ UD ゴシック」「Noto Sans Mono CJK JP」などの日本語対応フォントに変更してください。

第 17 まとめ

17.1 このマニュアルで学んだこと

このマニュアルを通じて、以下のことを学びました。

1. WSL の導入

Windows に Ubuntu をインストールし、コマンドラインで Linux を使えるようにしました。

2. ターミナルの基本操作

プロンプトの見方、ショートカットキー、Tab 補完などを学び、コマンドラインに慣れました。

3. Unix/Linux の考え方

ファイルシステムの階層構造、絶対パスと相対パス、/`~`...などのパス記号を理解しました。

4. 基本コマンドの習得

`pwd`, `ls`, `cd`, `mkdir`, `touch`, `cp`, `mv`, `rm`, `cat` を使ってファイル操作ができるようになりました。

5. Python 仮想環境

なぜ仮想環境が必要かを理解し、`venv` を使ってプロジェクトごとに独立した Python 環境を構築できるようになりました。

6. VS Code との連携

WSL と VS Code を連携させ、快適な開発環境を整えました。

17.2 次のステップ

環境が整ったら、次のことに挑戦してみましょう。

Python を深く学ぶ 変数、制御構造、関数、クラスなど Python の基礎を固めましょう。

授業のテキストや「Python チュートリアル (<https://docs.python.org/ja/3/tutorial/>)」が参考になります。

Jupyter Notebook を使う データ分析や機械学習の授業では Jupyter Notebook をよく使います。pip3 install jupyter でインストールし、VS Code から使えます。

Git を学ぶ コードのバージョン管理ツール Git を使うと、変更履歴の管理やチームでの共同作業ができます。sudo apt install git でインストールできます。

Bash スクリプトに挑戦する 繰り返し実行するコマンドをスクリプトファイルにまとめると、作業を自動化できます。

17.3 困ったときの参考先

- 第 16 章（よくある質問とトラブルシューティング）
- 付録 A（コマンド一覧）
- Microsoft WSL 公式ドキュメント：<https://docs.microsoft.com/ja-jp/windows/wsl/>
- Python 公式チュートリアル（日本語）：<https://docs.python.org/ja/3/tutorial/>
- Ubuntu 日本語フォーラム：<https://forums.ubuntulinux.jp/>

17.4 最後に

コマンドライン操作は最初は不思議で怖く感じるかもしれませんが、繰り返し使うことで自然に慣れていきます。エラーが出て焦らず、メッセージをよく読み、少しずつ解決していきましょう。

「わからない」「うまくいかない」は成長のチャンスです。このマニュアルを手元に置きながら、コマンドラインを使いこなせるようになってください。

付録 A よく使うコマンド一覧

A.1 ファイル・ディレクトリ操作

コマンド	由来	説明
<code>pwd</code>	Print Working Directory	現在のディレクトリのパスを表示
<code>ls</code>	LiSt	ディレクトリの内容一覧を表示
<code>ls -l</code>	—	詳細情報（パーミッション・サイズ・日時）付きで表示
<code>ls -la</code>	—	隠しファイルも含めて詳細表示
<code>cd ディレクトリ</code>	Change Directory	指定ディレクトリへ移動
<code>cd</code>	—	ホームディレクトリへ移動
<code>cd ~</code>	—	ホームディレクトリへ移動
<code>cd ..</code>	—	親ディレクトリへ移動
<code>cd -</code>	—	直前のディレクトリへ戻る
<code>mkdir 名前</code>	MaKe DIRectory	ディレクトリを作成
<code>mkdir -p a/b/c</code>	—	中間ディレクトリも含めて作成
<code>touch ファイル</code>	touch	空ファイルを作成（存在する場合はタイムスタンプ更新）

コマンド	由来	説明（続き）
cp 元 先	CoPy	ファイルをコピー
cp -r 元/ 先/	—	ディレクトリを再帰的にコピー
mv 元 先	MoVe	ファイルを移動・名前変更
rm ファイル	ReMove	ファイルを削除（元に戻せない）
rm -i ファイル	—	削除前に確認する（安全）
rm -r ディレクトリ	—	ディレクトリを再帰的に削除
cat ファイル	conCATenate	ファイルの内容を表示

A.2 ファイル閲覧

コマンド	由来	説明
less ファイル	less	ページ送りしながらファイルを表示（q で終了）
head ファイル	head	ファイルの先頭 10 行を表示
head -n 20 ファイル	—	先頭 20 行を表示
tail ファイル	tail	ファイルの末尾 10 行を表示
tail -f ファイル	—	ファイルの追記を監視し続ける
wc ファイル	Word Count	行数・単語数・文字数を表示
wc -l ファイル	—	行数のみ表示

A.3 システム・情報確認

コマンド	由来	説明
whoami	WHO AM I	現在のユーザー名を表示
clear	clear	画面をクリア (<code>Ctrl</code> + <code>L</code> でも可)
history	history	コマンド履歴を表示
which コマンド	which	コマンドの実体のパスを表示
exit	exit	ターミナルを終了
man コマンド	MANual	コマンドのマニュアルを表示
コマンド --help	—	コマンドの簡易ヘルプを表示

A.4 パッケージ管理 (apt)

コマンド	説明
sudo apt update	インストール可能なパッケージ一覧を更新
sudo apt upgrade	インストール済みパッケージを最新版に更新
sudo apt install パッケージ名	パッケージをインストール
sudo apt remove パッケージ名	パッケージをアンインストール
apt search キーワード	パッケージを検索

A.5 Python・仮想環境

コマンド	説明
python3 --version	Python バージョンを表示

コマンド	説明（続き）
<code>python3</code>	Python インタラクティブモードを起動
<code>python3 スクリプト.py</code>	Python スクリプトを実行
<code>pip3 --version</code>	pip バージョンを表示
<code>pip3 install パッケージ</code>	パッケージをインストール
<code>pip3 list</code>	インストール済みパッケージ一覧を表示
<code>pip3 freeze > requirements.txt</code>	パッケージ一覧をファイルに書き出す
<code>pip3 install -r requirements.txt</code>	requirements.txt からインストール
<code>python3 -m venv .venv</code>	仮想環境を作成
<code>source .venv/bin/activate</code>	仮想環境を有効化
<code>deactivate</code>	仮想環境を無効化

付録 B よく使うオプション一覧

B.1 ls のオプション

オプション	説明
-l	詳細情報（パーミッション、リンク数、所有者、サイズ、更新日時）付きで表示
-a	隠しファイル（. で始まるファイル）を含めて表示
-h	サイズを人間が読みやすい単位（K, M, G）で表示（-l と組み合わせる）
-r	逆順で表示
-t	更新日時の新しい順で表示
-S	ファイルサイズの大きい順で表示
-R	サブディレクトリも再帰的に表示
-1	1 行に 1 エントリで表示

よく使う組み合わせ：

コマンド	説明
ls -la	隠しファイルも含めた詳細表示
ls -lh	人間が読みやすいサイズの詳細表示
ls -lt	更新日時の新しい順で詳細表示
ls -lS	ファイルサイズが大きい順で詳細表示

B.2 cp のオプション

オプション	説明
-r	ディレクトリを再帰的にコピー（ディレクトリのコピーに必須）
-i	コピー先にファイルが存在する場合、上書き前に確認する
-u	コピー元がコピー先より新しい場合のみコピー
-v	実行内容を表示（verbose）
-p	パーミッション・タイムスタンプを保持してコピー

B.3 rm のオプション

オプション	説明
-i	削除前に確認する（interactive）。初心者強く推奨
-r	ディレクトリを再帰的に削除（ディレクトリの削除に必須）
-f	確認なしに強制削除（force）。注意して使うこと
-v	削除したファイルを表示（verbose）

危険

`rm -rf` は強力で元に戻せません。特にパスを間違えると重大な被害になります。初心者は `rm -i` を基本として使い、慣れるまで `-f` は使わないことを推奨します。

B.4 mkdir のオプション

オプション	説明
-p	中間ディレクトリが存在しなくても作成する。既に存在する場合もエラーにならない
-v	作成したディレクトリを表示（verbose）

B.5 head / tail のオプション

オプション	説明
-n N	先頭（または末尾）N 行を表示
-f (tail のみ)	ファイルの追記をリアルタイムで監視 (<code>Ctrl</code> + <code>C</code> で終了)

B.6 wc のオプション

オプション	説明
-l	行数のみ表示 (lines)
-w	単語数のみ表示 (words)
-c	バイト数のみ表示 (bytes)
-m	文字数のみ表示 (characters、マルチバイト対応)

付録 C 用語集

apt (Advanced Package Tool) Ubuntu/Debian 系 Linux のパッケージ管理ツール。ソフトウェアのインストール・更新・削除を行うために使う。`sudo apt install パッケージ名` の形式で使用する。

Bash (Bourne Again SHell) Linux の標準的なシェル。ユーザーが入力したコマンドを受け取り、OS に伝えるプログラム。Ubuntu のデフォルトシェル。

カレントディレクトリ (current directory) 現在いる (作業している) ディレクトリ。 `.` で表される。 `pwd` で確認できる。

GUI (Graphical User Interface) マウスでアイコンをクリックするなど、グラフィックで操作するインターフェース。Windows のデスクトップ環境など。CLI (コマンドラインインターフェース) の対義語。

CLI (Command Line Interface) 文字を入力してコンピュータを操作するインターフェース。ターミナルで使う。

ディストリビューション (distribution) Linux のカーネルに様々なソフトウェアを組み合わせたパッケージ。Ubuntu, Debian, Fedora, Arch Linux などがある。このマニュアルでは Ubuntu を使用する。

ディレクトリ (directory) フォルダのこと。コマンドラインや技術文書では「ディレクトリ」と呼ぶ。

ホームディレクトリ (home directory) ログインしたユーザー専用の作業スペース。Linux では `/home/ユーザー名` に位置する。 `~` で短縮表記できる。

パーミッション (permission) ファイルやディレクトリへのアクセス権限。読み取り (read, r)、書き込み (write, w)、実行 (execute, x) の 3 種類がある。 `ls -l` で確認できる。

パス (path) ファイルやディレクトリの「住所」を表す文字列。絶対パス (`/` から始まる完全パス) と相対パス (カレントディレクトリからの相対位置) がある。

パイプ (pipe, `|`) コマンドの出力を別のコマンドの入力として渡す仕組み。例: `history | grep python` で、`history` の出力から `python` を含む行を抽出する。

pip (Pip Installs Packages) Python のパッケージ管理ツール。PyPI からパッケージをインストールする。Ubuntu では `pip3` コマンドを使う。

プロンプト (prompt) ターミナルで次のコマンドの入力を待っていることを示す文字列。

通常は **ユーザー名@ホスト名: ディレクトリ\$** の形式。

PyPI (Python Package Index) Python パッケージが公開されているリポジトリ (倉庫)。

`pip3 install` はここからダウンロードする。<https://pypi.org/> で検索できる。

ルートディレクトリ (root directory) Linux ファイルシステムの最上位のディレクトリ。

`/` で表される。すべてのファイルはここから始まるツリー構造の中にある。

ルートユーザー (root user) すべての権限を持つ管理者アカウント。Linux では `root` というユーザー名。通常の作業では使わず、`sudo` で一時的に管理者権限を使う。

シェル (shell) ユーザーが入力したコマンドを OS (カーネル) に伝えるプログラム。Bash の他に Zsh, Fish などがある。

シンボリックリンク (symbolic link) ファイルやディレクトリへの参照 (ショートカットのようなもの)。仮想環境の `.venv/bin/python3` は、実際の Python へのシンボリックリンクであることが多い。

sudo (Superuser DO) 管理者 (スーパーユーザー) 権限でコマンドを実行するコマンド。システムに影響する操作 (`apt install` など) に必要。パスワードを求められることがある。

ターミナル (terminal) コマンドを入力してコンピュータを操作するためのアプリ・インターフェース。このマニュアルでは主に Windows Terminal を使用する。

仮想環境 (virtual environment) プロジェクトごとに独立した Python 実行環境。パッケージのバージョンを分離して管理できる。Python 標準の `venv` や、高速な `uv` などのツールで作成できる。

WSL (Windows Subsystem for Linux) Windows 上で Linux を動かす仕組み。Microsoft が公式に提供している。現在の主流バージョンは WSL2。

WSL2 WSL のバージョン 2。Hyper-V を利用した実際の Linux カーネルを使用しており、WSL1 より高い互換性とパフォーマンスを持つ。

付録 D 参考資料

D.1 公式ドキュメント

Microsoft WSL 公式ドキュメント (日本語) <https://docs.microsoft.com/ja-jp/windows/wsl/> — WSL のインストール方法、設定、トラブルシューティングなど。

Ubuntu 公式ドキュメント <https://ubuntu.com/server/docs> — Ubuntu の公式マニュアル (英語)。

Python 公式チュートリアル (日本語) <https://docs.python.org/ja/3/tutorial/> — Python の基礎から応用まで網羅した公式ドキュメント。

Python 仮想環境公式ドキュメント (日本語) <https://docs.python.org/ja/3/library/venv.html> — venv の詳細な使い方。

pip 公式ドキュメント <https://pip.pypa.io/en/stable/> — pip の詳細な使い方 (英語)。

uv 公式ドキュメント <https://docs.astral.sh/uv/> — uv の詳細な使い方とリファレンス (英語)。

VS Code 公式ドキュメント (WSL の使い方) <https://code.visualstudio.com/docs/remote/wsl> — VS Code と WSL を連携させる方法 (英語)。

D.2 Linux コマンドの参考サイト

Linux コマンド集 (IT 専科) 日本語で Linux コマンドの使い方を説明したサイト。

tldr pages <https://tldr.sh/> — コマンドの使い方を簡潔な例で示すリファレンス。tldr ls のように使える (要インストール)。

ExplainShell <https://explainshell.com/> — シェルコマンドを貼り付けると、各部分の意味を説明してくれるサイト (英語)。

D.3 学習リソース

The Missing Semester of Your CS Education (日本語訳あり) <https://missing.csail.mit.edu/> — MIT のコンピュータサイエンス学生向けのコマンドライン・

ツール入門。シェル、スクリプト、Git、デバッグなど。

Linux の教科書 Linux のコマンドラインを体系的に学べる入門書。書籍版もある。

付録 E 演習課題

E.1 基礎演習：コマンドライン操作

演習 1：ディレクトリ操作の基本

以下の手順を実行し、各ステップで画面の出力を確認してください。

1. ホームディレクトリに移動し、現在位置を確認する
2. `exercises` というディレクトリを作成する
3. `exercises` の中に `week01`、`week02`、`week03` を一括作成する
4. `exercises` に移動し、内容を確認する
5. `week01` の中に `notes.txt`、`exercise.py` を作成する
6. `exercises` に戻り、ディレクトリ全体を `ls -R` で確認する

メモ

確認コマンド：

```
$ ls -R ~/exercises/
```

以下のような出力になれば成功です：

```
/home/taro/exercises/:
```

```
week01  week02  week03
```

```
/home/taro/exercises/week01:
```

```
exercise.py  notes.txt
```

```
/home/taro/exercises/week02:
```

```
/home/taro/exercises/week03:
```

演習 2：ファイルのコピーと移動

演習 1 で作成した `exercises` ディレクトリで以下を行ってください。

1. `week01/notes.txt` を `week02/notes.txt` としてコピーする
2. `week01/exercise.py` を `week03/exercise.py` に移動する
3. 各ディレクトリの内容を `ls` で確認する
4. `week03` の中に `archive` ディレクトリを作り、`week03/exercise.py` を移動する
5. 最後に `ls -R /exercises/` で全体を確認する

演習 3：パスの理解

ホームディレクトリ (`~`) にいる状態で、以下のコマンドが何を意味するか答えてください。
(実際に実行して確認してください)

1. `ls .` は何を表示するか
2. `ls ..` は何を表示するか
3. `cd exercises/week01` と `cd /exercises/week01` の違いは何か
4. `exercises/week01` にいるとき、`cd ../../week02` の操作でどこに移動するか

E.2 中級演習：Python 仮想環境

演習 4：仮想環境の作成と使用

1. `~/myproject` というディレクトリを作成して移動する
2. `.venv` という名前の仮想環境を作成する
3. 仮想環境を有効化し、プロンプトの変化を確認する
4. 仮想環境内の Python と pip のパスを `which` で確認する
5. `numpy` をインストールする
6. `pip3 list` でインストールされたことを確認する
7. `requirements.txt` に書き出す
8. 仮想環境を無効化する
9. 無効化後に `which python3` で Python のパスが変わることを確認する

演習 5：仮想環境から Python スクリプトを実行する

1. 演習 4 で作成した `myproject` に移動し、仮想環境を有効化する
2. 以下の内容の `check.py` を作成する

Listing E.1 check.py

```
import numpy as np
import sys

print("Python バージョン:", sys.version)
print("NumPy バージョン:", np.__version__)
a = np.array([1, 2, 3, 4, 5])
print("配列:", a)
print("合計:", a.sum())
print("平均:", a.mean())
```

3. python3 check.py を実行して出力を確認する
4. 仮想環境を無効化してから同じスクリプトを実行し、何が起きるか確認する

E.3 発展演習

演習 6：コマンドを組み合わせる

以下の操作をコマンド 1 行で実現してください。

1. ~/exercises/week01/ にある .txt ファイルの行数を数える (wc とファイルパスを使う)
2. コマンド履歴から python を含む行だけを表示する (history と | と grep を使う)
3. exercises 以下にある .py ファイルを find コマンドで一覧表示する

```
$ find ~/exercises -name "*.py"
```

演習 7：仮想環境の復元

1. myproject の .venv ディレクトリを削除する (rm -r .venv)
2. requirements.txt から仮想環境を再作成・復元する
 - python3 -m venv .venv
 - source .venv/bin/activate
 - pip3 install -r requirements.txt
3. 復元後に check.py が再び実行できることを確認する

ポイント

演習のポイント：

- エラーが出ても焦らず、メッセージをよく読む

- わからなくなったら `pwd` で現在位置を確認する
- `--help` や `man` でコマンドの使い方を確認する
- 大事なファイルは削除前に確認する (`rm -i` を使う)